

# PROJECT SOUNDING LINE

The Software Verification and Parallel Test Infrastructure System

## PART III

# Repository Policy, Codex Governance, and Release Assurance

Governing Architecture and Product Requirements Document

### Governing Principle

The repository must require the same verification truth from humans, Codex, CI, integration work, and releases. Every behavior change must declare its affected contracts and test obligations; every omission, retry, quarantine, debt item, exception, and release status must be explicit; and no completion claim may outrun the evidence produced by Parts I and II.

# Document Control

| Field                    | Value                                                                                                                                                                                                                                                                                                                                                                       |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Program                  | Project Sounding Line                                                                                                                                                                                                                                                                                                                                                       |
| Subsystem                | The Software Verification and Parallel Test Infrastructure System                                                                                                                                                                                                                                                                                                           |
| Document                 | Governing Architecture and Product Requirements Document - Part III                                                                                                                                                                                                                                                                                                         |
| Part III scope           | Developer and Codex obligations, mandatory change-to-test rules, negative and regression policy, stale tests, assertion quality, release gates, branch and mainline policy, release decisions, flakes, quarantine, validation debt, evidence retention, security/privacy, performance budgets, policy files, incidents, exceptions, rollout, training, and program closure. |
| Companion document       | Part I - Software Verification Architecture                                                                                                                                                                                                                                                                                                                                 |
| Companion document       | Part II - Execution Infrastructure and Parallel Runtime                                                                                                                                                                                                                                                                                                                     |
| Version                  | 1.0                                                                                                                                                                                                                                                                                                                                                                         |
| Date                     | July 28, 2026                                                                                                                                                                                                                                                                                                                                                               |
| Status                   | Governing baseline                                                                                                                                                                                                                                                                                                                                                          |
| Document ID              | CS-SL-P3-001                                                                                                                                                                                                                                                                                                                                                                |
| Primary surfaces         | Developer work, Codex tasks, pull requests, integration branches, mainline convergence, CI, release candidates, external-provider validation, incident response, and program governance.                                                                                                                                                                                    |
| Core implementation rule | The repository must enforce Part I proof semantics through the Part II runtime; behavior and tests evolve together; exceptions are explicit and temporary; release wording is evidence-bound.                                                                                                                                                                               |
| Repository relationship  | The current fetched repository remains implementation truth. Accepted governing documents define obligations and boundaries; historical SHAs and completion reports are evidence, not restoration commands.                                                                                                                                                                 |

## Three-document split

**Part I** defines verification truth. **Part II** executes that truth. **Part III** enforces it across humans, Codex, CI, branches, and releases. Part III may require the runtime and evidence model to be used; it may not bypass or redefine them.

## Normative authority

Part III is authoritative for repository testing obligations, source-to-test maintenance, Codex behavior, test review, gate use, branch and mainline requirements, completion language, exception and debt governance, evidence retention, security and privacy policy for tests, performance budgets, incident handling, policy validation, rollout, audit, adoption, training, and final Sounding Line closure.

# Contents

|                                           |                                                                 |
|-------------------------------------------|-----------------------------------------------------------------|
| 1. Governing Policy Hierarchy             | 25. Repository Status Integration                               |
| 2. Developer Testing Obligations          | 26. Codex Prompt Contract                                       |
| 3. Codex Testing Obligations              | 27. Codex Completion Report Format                              |
| 4. Mandatory Change-to-Test Rules         | 28. Incident Policy                                             |
| 5. Negative Test Policy                   | 29. Exception Policy                                            |
| 6. Regression-Test Policy                 | 30. Governance Changes Requiring ADR                            |
| 7. Stale-Test Policy                      | 31. Implementation Roadmap Detail                               |
| 8. Assertion-Quality Policy               | 32. Adoption Plan                                               |
| 9. Test-Review Checklist                  | 33. Training and Usage Guidance                                 |
| 10. Release-Gate Classes                  | 34. Final Program Acceptance Criteria                           |
| 11. Mainline Policy                       | Appendix A. Normative Change-to-Test Matrix                     |
| 12. Release Decision Policy               | Appendix B. Codex Preflight and Completion Checklists           |
| 13. Flake Policy                          | Appendix C. Gate Evidence Matrix                                |
| 14. Quarantine Policy                     | Appendix D. Flake, Quarantine, Debt, and Exception Registers    |
| 15. Validation-Debt Policy                | Appendix E. Machine-Readable Policy Schemas                     |
| 16. Evidence-Retention Policy             | Appendix F. Pull Request and Integration Templates              |
| 17. Security and Privacy Policy for Tests | Appendix G. Release Decision and Completion Wording Catalog     |
| 18. Test-Data Governance                  | Appendix H. Incident Classification and Recovery Playbooks      |
| 19. Performance Budgets                   | Appendix I. Performance Budget and Slow-Suite Review Worksheets |
| 20. Slow-Suite Governance                 | Appendix J. Roles, Authorities, and Separation of Duties        |
| 21. Historical Result Governance          | Appendix K. Policy Audit and Compliance Scenarios               |
| 22. Documentation Requirements            | Appendix L. Four-Phase Governance Implementation Map            |
| 23. Machine-Readable Policy Files         | Appendix M. Part III Normative Completion Checklist             |
| 24. Policy Validation                     | References and Final Governing Rule                             |

## How to read this document

Sections 1-9 govern everyday change and test quality. Sections 10-18 govern gates, release language, instability, evidence, privacy, and data. Sections 19-27 govern performance, policy files, repository integration, and Codex contracts. Sections 28-34 govern incidents, exceptions, program rollout, training, and closure. The appendices provide normative matrices, templates, schemas, registers, and checklists.

## PART III

# 1. Governing Policy Hierarchy

Part III is the repository constitution that turns the architecture of Part I and the runtime of Part II into mandatory behavior. It governs developers, Codex, integration owners, CI, release managers, and infrastructure maintainers. It does not invent new test meanings or execution mechanics. It defines which authorities prevail, how conflicts are resolved, when exceptions exist, and what evidence is required before a change may advance.

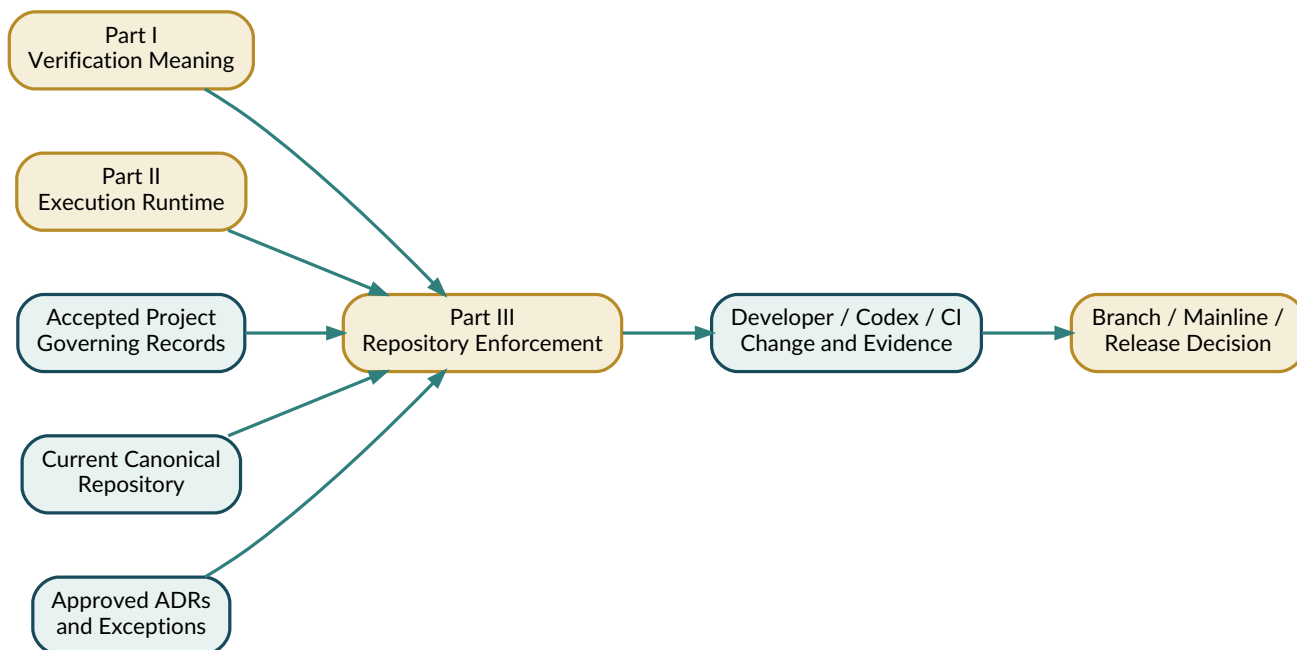


Figure 1. Normative policy hierarchy from governing architecture and current repository state to enforceable change and release decisions.

## 1.1 Precedence order

| Priority | Authority                                                          | Use                                                                                                     |
|----------|--------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| 1        | Accepted Project Sounding Line Parts I, II, and III                | Verification meaning, execution contracts, repository obligations, and release assurance.               |
| 2        | Accepted cross-project governing documents                         | Product semantics, ownership, privacy, progression, providers, and project-specific completion gates.   |
| 3        | Current fetched canonical repository                               | Actual implementation paths, versions, scripts, migrations, dependencies, and accepted source behavior. |
| 4        | Approved architecture decision records and governed amendments     | Explicit refinements, exceptions, migrations, and technology choices.                                   |
| 5        | Accepted completion, validation, integration, and incident records | Evidence of what was implemented and proven at a specific source state.                                 |
| 6        | Historical audits, roadmaps, prompts, and branch records           | Context and failure history; never restoration authority by themselves.                                 |
| 7        | Chats, drafts, informal notes, and recollection                    | Non-authoritative planning material until accepted into governed records.                               |

## 1.2 Conflict resolution

- First determine whether the conflict concerns product meaning, verification meaning, execution mechanics, repository enforcement, or current implementation reality.
- The owning governing authority resolves meaning; current repository paths and versions resolve implementation location only when they do not violate the governing invariant.

- A newer accepted amendment may supersede an older document, but the supersession must identify the exact affected clauses and migration consequences.
- When two safe interpretations remain materially different, the change is blocked before irreversible implementation or release action.
- Ordinary naming, file layout, adapter, and scheduling choices do not require owner intervention when Parts I-III already define the invariant.
- A source-code accident, stale test, or optimistic completion message never silently amends policy.

### 1.3 Amendment and emergency authority

| Mechanism              | Permitted use                                                                          | Mandatory evidence                                                                                       |
|------------------------|----------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| Governing amendment    | Durable change to verification, runtime, or repository policy.                         | Proposal, affected clauses, risk analysis, migration, compatibility, tests, approver, effective version. |
| ADR                    | Technology or architecture choice within existing governing boundaries.                | Options, decision, consequences, rollback, owner, acceptance tests.                                      |
| Time-bounded exception | Temporary deviation when the governing path cannot be completed immediately.           | Scope, reason, risk, compensating evidence, expiry, owner, approver, release effect.                     |
| Emergency containment  | Immediate action to protect data, credentials, repository state, or release integrity. | Incident ID, preserved evidence, authority, bounded action, recovery and review.                         |

#### Hard Boundary

No emergency exception may redefine a failed, blocked, skipped, external-pending, or invalid-evidence outcome as a pass. It may authorize containment or a bounded release decision only with explicit residual risk and authority.

## PART III

## 2. Developer Testing Obligations

Every developer is responsible for the verification consequences of a behavioral change. Test ownership may be distributed, but test impact cannot be delegated to an imaginary future integrator. A change is not ready for review until its intended behavior, affected contracts, required evidence, and known gaps are stated clearly enough for another engineer to reproduce the decision.



Figure 2. Required developer change lifecycle: intent, impact, source and test maintenance, evidence, and completion receipt.

### 2.1 Required change declaration

| Declaration        | Required content                                                                                                                             |
|--------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| Behavior statement | What user-visible, authoritative, security, persistence, provider, or operational behavior changes and what remains intentionally unchanged. |
| Ownership          | Primary source owner, contract owner, test owner, infrastructure owner, and escalation owner when boundaries are crossed.                    |
| Contracts          | Contract IDs added, modified, superseded, or consumed; versioning and compatibility effect.                                                  |
| Impact             | Direct files, transitive consumers, schema/routes, security/privacy, browser/device, migrations, historical readers, and providers.          |
| Test obligations   | Suites to add or update, required tiers, negative cases, migration/restart/replay coverage, and release-gate membership.                     |
| Evidence status    | What has run, what has not run, why, freshness, environment, cleanup, debt, quarantine, and external gates.                                  |

### 2.2 Same-task source and test rule

- Behavior changes and the tests protecting that behavior are updated in the same branch and ordinary task.
- A missing test is an implementation gap, not documentation debt, unless the gate explicitly permits a time-bounded validation-debt record.
- Test metadata, contract mappings, fixture versions, and impact rules are updated when the behavior change affects them.
- Intentional removal of behavior removes or migrates its tests only after the replacement contract and retirement evidence exist.
- Documentation-only edits need not invent product tests, but policy, examples, generated indexes, links, and document build checks may still be impacted.
- Infrastructure changes include tests for isolation, cleanup, failure classification, and rollback rather than only happy-path command execution.

### 2.3 Developer pre-review checklist

| Question                                                   | Closure rule                                                     |
|------------------------------------------------------------|------------------------------------------------------------------|
| Can another person state the changed behavior?             | If not, the change declaration is incomplete.                    |
| Are affected contracts and owners identified?              | Unknown ownership broadens the plan or blocks closure.           |
| Do tests fail before the repair where practical?           | Bug fixes ordinarily require a reproducer.                       |
| Are negative and failure paths addressed?                  | Non-applicable categories require explicit explanation.          |
| Are tests isolated and parallel-safe under their metadata? | Unclassified mutable resources require serial or blocked status. |
| Is evidence tied to the exact source and policy?           | Unbound console output is not accepted evidence.                 |
| Did cleanup complete without touching canonical state?     | A passing suite with failed cleanup is not complete.             |

**Developer obligation**

A pull request that changes behavior but contains no reasoned test-impact statement is incomplete even when the obvious tests pass. Passing the nearest test is not the same as proving the changed contract.

## PART III

## 3. Codex Testing Obligations

Codex is treated as an implementation agent subject to the same verification policy as a human developer, plus additional controls for autonomy, durability, worktree safety, and completion wording. Codex may make ordinary engineering decisions, run task-owned tools, and continue through routine failures. It may not weaken tests, escape ownership, or convert inconvenience into a blocker.

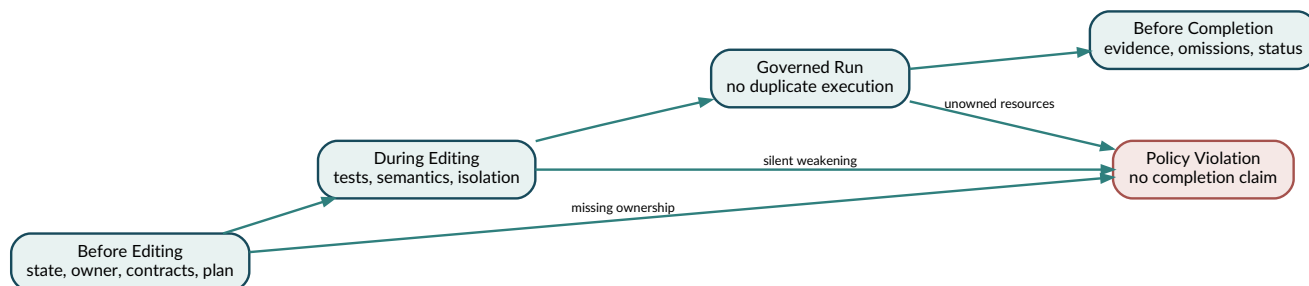


Figure 3. Codex governance lifecycle with preflight, same-task testing, governed execution, and completion evidence.

### 3.1 Before editing

- Fetch or otherwise verify the current repository state required by the task; record branch, commit, upstream, worktree, staged, unstaged, and untracked state.
- Identify source owner, contract owner, test owner, and any active concurrent work affecting shared files, migrations, ports, databases, or validation infrastructure.
- State the intended behavior change, non-goals, likely contracts, risk class, expected plan, and required negative cases.
- Inspect current tests and fixtures before creating replacements; preserve newer accepted behavior and avoid restoring stale baseline files.
- Create or use a task-owned branch, worktree, runtime roots, ports, databases, accounts, and storage as the governing task requires.
- Do not treat missing dependencies, an unset local database URL, a busy unrelated gate, or one failing test as an automatic stopping condition.

### 3.2 During editing

| Codex must                                                           | Codex must not                                                             |
|----------------------------------------------------------------------|----------------------------------------------------------------------------|
| Add or update tests and metadata in the same task.                   | Delete assertions without preserving the governed risk.                    |
| Use semantic, authoritative, and user-visible assertions.            | Replace contract proof with generic visibility or snapshot theater.        |
| Start with focused reproduction and expand according to impact.      | Run the complete repository repeatedly because planning was skipped.       |
| Preserve unrelated staged, unstaged, untracked, and concurrent work. | Use destructive Git commands, broad cleanup, or touch another worktree.    |
| Use governed durable runs and inspect existing run identity.         | Launch duplicate browser/build/release runs because a shell appears quiet. |
| Diagnose timeouts, races, and flakes.                                | Add arbitrary sleeps, broad retries, or higher timeouts without evidence.  |
| Continue independent work when one gate is externally blocked.       | Declare the entire task blocked while locally attainable work remains.     |

### 3.3 Before completion

- Reconcile changed files, changed behaviors, contracts, tests added, tests updated, tests run, tests omitted, and reasons.
- State focused, integration, compatibility, release, and external-provider evidence separately.
- Report first-pass failures, retries, quarantines, debt, cleanup problems, and invalidated evidence rather than smoothing the story.
- Use Part I validation statuses exactly; do not call a branch release complete when only focused evidence exists.
- Commit and push only reviewed task-owned changes, verify remote parity where required, and leave task-owned resources in a governed terminal state.



- Do not send an interim progress report as a completion response while substantial locally attainable scope remains.

## 3.4 Prohibited completion behavior

| Prohibited behavior                                       | Why it is invalid                                                 |
|-----------------------------------------------------------|-------------------------------------------------------------------|
| "Implementation complete" after one unit family           | Implementation breadth and required evidence are unknown.         |
| "Blocked" because node_modules or DATABASE_URL is missing | Ordinary local setup remains Codex responsibility.                |
| Timeout increase without root cause                       | Hides performance or synchronization defects.                     |
| test.only, unconditional skip, or blanket retry           | Changes the proof without approval.                               |
| Killing unrelated Node/browser processes                  | Violates Part II ownership and can corrupt concurrent work.       |
| Declaring release GO from a feature branch gate           | Focused or phase evidence cannot promote itself to release proof. |

## PART III

## 4. Mandatory Change-to-Test Rules

Change-to-test rules define the minimum verification floor by change type. The planner may expand beyond these floors through contracts, risk, dependencies, release gates, or uncertainty. These rules are not a substitute for impact analysis; they prevent common changes from slipping through with only the nearest convenient test.

### 4.1 Core change classes

| Change class               | Minimum required evidence                                                                                                                                                                                                                      |
|----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| UI behavior                | Tier 1 logic where applicable; Tier 2 component, keyboard, focus, loading/error states; Tier 4 focused browser for journey changes; responsive proof for layout changes; reduced motion when motion changes.                                   |
| API contract               | Request/response schema, validation, authentication, authorization, CSRF for browser mutation, stale state, idempotency where relevant, error redaction, service/database integration.                                                         |
| Database/schema            | Both schema definitions where applicable, formatting/validation/generation, additive migration, empty rehearsal, upgrade rehearsal, foreign keys/integrity, service/projection regressions, MySQL preparation or live proof according to gate. |
| Authentication/session     | Unauthorized, expired, revoked, wrong-role, cross-account, replay, CSRF, rate-limit, session rotation/revocation, browser journey, privacy and logging.                                                                                        |
| Animation/presentation     | Scene/target contract, lifecycle, remount, interruption, replay, reduced motion, fallback, semantic success, accessibility, browser proof, cleanup.                                                                                            |
| Privacy/public projection  | Allowlist projection, IDOR, foreign account, logs/traces/screenshots, public/private boundary, staged privacy scan, search/OG/cache exclusion.                                                                                                 |
| Geolocation/provider       | Permission denial, stale/low-accuracy evidence, unavailable provider, offline, privacy retention, manual or accessible fallback, duplicate evidence, authoritative progression boundary.                                                       |
| Private content/security   | Malformed input, authorization, package/integrity, scanner unavailable, quarantine, secret redaction, rollback, backup/restore or provider fault where relevant.                                                                               |
| Build/dependency/toolchain | Lockfile identity, clean install, generated outputs, production build, affected browser/runtime suites, supply-chain scan, compatibility and rollback.                                                                                         |
| Documentation/policy       | Link/path validity, generated indexes, schema examples, policy validator, document build/render, amendment and supersession references.                                                                                                        |

### 4.2 Escalation rules

- Authorization, privacy, data-loss, migration, public projection, private media, progression, backup/restore, and security changes escalate to at least HIGH risk.
- Changes to shared schemas, package lock, runtime policy, test infrastructure, global navigation, or canonical projection broaden to cross-project evidence.
- Changes affecting historical readers or immutable published versions require compatibility and migration fixtures.
- Changes to mobile layout, PageFlip, gesture handling, map/camera, or media renderers require representative browser/device matrices.
- External-provider adapter changes require deterministic local contracts even when live credentials are unavailable; live status remains separate.
- Unknown change class or missing ownership expands to a conservative plan rather than defaulting to no tests.

### 4.3 Plan explanation requirement

Selected because: <direct owner / contract / dependency>  
 Expanded because: <risk / schema / security / compatibility / release gate>  
 Required negative cases: <list or explicit non-applicability>  
 Omitted because: <governed reason>  
 External only: <provider/device and status>  
 Deferred with debt: <debt ID, owner, expiry, release effect>

**Minimum is not maximum**

Meeting the row for a UI change does not permit omitting an affected authorization contract, migration, or cross-project consumer. The matrix establishes floors; the impact graph establishes the actual plan.

## PART III

## 5. Negative Test Policy

Negative testing is required whenever the system can reject, deny, time out, retry, recover, degrade, or expose information incorrectly. Positive-only proof demonstrates that one intended path can work. It does not prove that the platform refuses unsafe paths or remains understandable when ordinary reality behaves badly.

### 5.1 Negative category catalog

| Category                   | Representative obligations                                                                                                        |
|----------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Invalid input              | Missing, malformed, out-of-range, oversized, unknown enum, incompatible fields, unsafe path, unsupported version.                 |
| Authorization              | Unauthenticated, wrong role, foreign account, removed membership, suspended account, expired invitation, guessed identifier.      |
| Stale and concurrent state | Old revision, duplicate request, reordered event, conflicting update, lost lease, retry after authoritative commit.               |
| Provider and environment   | Unavailable, degraded, timeout, malformed response, stale health, credential denial, wrong namespace, offline.                    |
| Lifecycle                  | Cancel before start, cancel during side effect, unmount, route change, process crash, restart, cleanup failure, resume.           |
| Privacy                    | Redacted error, safe not-found, no secret logs, no hidden DOM payload, no trace/screenshot leak, public projection allowlist.     |
| Accessibility              | No motion, no audio, keyboard-only, zoom, screen-reader state, visible focus, static fallback.                                    |
| Data integrity             | Corrupt database/object, checksum mismatch, migration drift, missing relation, duplicate grant, partial commit, restore mismatch. |
| Performance limits         | Bounded pagination, maximum payload, excessive entries, long path, large fixture, resource pressure, no-progress timeout.         |

### 5.2 Applicability declaration

Every material feature or bug fix must either cover the relevant categories or record why a category cannot affect the changed contract. “Not applicable” is a reasoned statement, not an empty checkbox. The explanation becomes part of the impact manifest and may be reviewed by the contract or gate owner.

### 5.3 Negative proof quality

- The test must prove the system refused, degraded, recovered, or redacted for the intended reason.
- An HTTP status alone is insufficient when the authoritative state, audit record, retry behavior, or public projection could still be wrong.
- A thrown exception is not proof of fail-closed behavior when a partial database or provider side effect may remain.
- Failure presentations must be accessible and must not animate or sound like success.
- Sensitive fixtures use synthetic data and redacted artifacts; security tests do not justify leaking the secret they are meant to protect.
- Negative tests remain deterministic and isolated; deliberate faults are scoped to the owning run.

#### Governing rule

When a dangerous failure mode is credible and untested, the absence of a convenient fixture is a test-infrastructure obligation, not evidence that the failure is impossible.

## PART III

## 6. Regression-Test Policy

A repaired defect should become a durable contract, not an anecdote. Regression tests preserve the smallest trustworthy reproduction, the semantic risk, and the affected family so the same failure cannot quietly return under a slightly different wrapper.

### 6.1 Ordinary regression sequence

| Step             | Requirement                                                                                                 |
|------------------|-------------------------------------------------------------------------------------------------------------|
| 1. Reproduce     | Create or identify a focused failing test or deterministic fixture at the pre-repair source when practical. |
| 2. Confirm cause | Show that the failure represents the governed defect rather than unrelated environment or fixture damage.   |
| 3. Repair        | Change the smallest responsible production and infrastructure scope without weakening adjacent contracts.   |
| 4. Prove focused | The reproducer passes and still fails when the repair is removed or fault is reintroduced where feasible.   |
| 5. Expand impact | Run affected family, contract consumers, negative paths, and required integration/compatibility nodes.      |
| 6. Record        | Link defect, cause, regression test, source change, evidence, and any remaining limitation.                 |

### 6.2 Exceptions to committed reproducer

| Exception                               | Required substitute evidence                                                                                          |
|-----------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| Security-sensitive exploit              | Private protected reproducer, sanitized public test, threat record, and authorized evidence retention.                |
| External provider outage                | Provider contract/fault simulation plus live incident evidence and future external gate.                              |
| Nondeterministic infrastructure failure | Repeated signature evidence, Part II fault injection, owner, flake/incident record, and deterministic invariant test. |
| Impossible historical environment       | Frozen historical artifact, compatibility reader test, documented limitation, and release effect.                     |
| Privacy-restricted personal content     | Synthetic equivalent preserving structure and size without private prose or media.                                    |

### 6.3 Regression ownership

- The production contract owner owns the semantic risk; the test owner maintains the suite; infrastructure failures retain an infrastructure owner.
- A regression test may migrate between tiers or routes when architecture changes, but the protected risk and stable contract reference remain.
- Retiring a regression requires evidence that the protected behavior no longer exists or is fully covered by a stronger accepted contract.
- A bug fix without a regression explanation cannot be promoted beyond IMPLEMENTED unless an authorized exception exists.

#### No ceremonial bug fixes

A completion report saying “the issue was fixed manually” is not regression evidence. The repository must retain a reproducible reason the defect stays fixed after everyone forgets the dramatic afternoon when it first appeared.

## PART III

## 7. Stale-Test Policy

Tests become stale when their asserted route, identity model, terminology, persistence authority, fixture, or implementation detail no longer represents the accepted product contract. Staleness is not permission to delete the test. It is a migration obligation: preserve the protected risk while adapting the test to the current authority.

### 7.1 Stale-test classifications

| Class                       | Typical symptom                                                                             | Required action                                                               |
|-----------------------------|---------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| TERMINOLOGY_STALE           | Old role, route, or product term remains in labels/assertions.                              | Update language and retain semantic behavior checks.                          |
| ROUTE_STALE                 | Journey targets a retired compatibility URL or page.                                        | Move to canonical route or verified adapter while preserving journey purpose. |
| SESSION_MODEL_STALE         | Fixture assumes retired token, cookie, account, or membership authority.                    | Migrate to canonical identity/session contract and preserve auth risks.       |
| PERSISTENCE_STALE           | Test reads retired mirror table or mutable field instead of authoritative event/projection. | Assert canonical source and historical compatibility where required.          |
| CONTRACT_SUPERSEDED         | Behavior was intentionally replaced by an accepted contract.                                | Map old risk to replacement suites and record supersession.                   |
| FIXTURE_STALE               | Seed or synthetic state no longer satisfies schema or lifecycle.                            | Version and migrate fixture; do not weaken production validation.             |
| IMPLEMENTATION_DETAIL_STALE | Test depends on wrapper, DOM identity, internal timing, or incidental CSS.                  | Replace with semantic/user-visible/authoritative assertion.                   |

### 7.2 Migration invariants

- State the original risk and the new authoritative surface before editing the test.
- Preserve authorization, persistence, idempotency, lifecycle, focus, accessibility, privacy, and failure coverage that the old test intended.
- Do not use architecture migration as an excuse to collapse a real browser journey into a shallow component snapshot.
- When the old behavior remains a supported compatibility surface, retain a compatibility suite with an explicit removal gate.
- Update suite metadata, contract mappings, owner, risk, resource profile, and historical fixture references in the same change.
- Record stale-test repairs separately from product defects so failure history remains intelligible.

### 7.3 Example: Quartermaster to Captain authority

A Quartermaster-specific test may migrate to the canonical Captain Console, but it must continue proving the same meaningful risks: authorized actor resolution, correct Voyage scope, stale command rejection, idempotent execution, durable state, focus and accessible feedback, and no privilege broadening. Replacing the test with “Captain page is visible” would be a silent deletion of nearly every contract.

#### Stale does not mean useless

A stale test often contains valuable failure history. The task is to separate the enduring risk from the obsolete mechanics, not to throw both into the sea because the selector became inconvenient.

## PART III

## 8. Assertion-Quality Policy

Assertions are ranked by the quality of evidence they provide. Lower-level implementation assertions remain useful for narrow units, but release confidence should prefer semantic behavior, authoritative persistence, user-visible outcomes, and explicit cross-system contracts over incidental structure.

### 8.1 Assertion quality levels

| Level                          | Description                                                                          | Typical use                                                              | Release value                       |
|--------------------------------|--------------------------------------------------------------------------------------|--------------------------------------------------------------------------|-------------------------------------|
| A0 - Incidental implementation | Exact wrapper, internal method, CSS class, arbitrary timeout, private function call. | Narrow implementation tests only.                                        | Low; brittle and non-semantic.      |
| A1 - Structural/DOM identity   | Specific element hierarchy, attribute, node count, or generated structure.           | Accessibility and renderer contracts when structure is itself normative. | Moderate when justified.            |
| A2 - Visual presence/state     | Visible text, control, loading, error, disabled, selected, layout state.             | Component and browser presentation.                                      | Useful but not authoritative alone. |
| A3 - Semantic user behavior    | Keyboard, focus, navigation, task outcome, understandable fallback.                  | User-facing acceptance.                                                  | High.                               |
| A4 - Authoritative persistence | Database event, projection, version pin, audit, idempotency, restart state.          | Service/runtime correctness.                                             | High and often mandatory.           |
| A5 - Cross-system contract     | Producer and consumer agree across project boundary under failure and compatibility. | Integration and release gates.                                           | Release critical.                   |

### 8.2 Assertion design rules

- Name the behavior and failure mode being proven; avoid tests whose title only repeats a component filename.
- Prefer stable role, accessible name, semantic state, event type, projection DTO, or contract ID over layout ancestry.
- Use authoritative state to confirm a mutation rather than trusting a transient success toast.
- Verify absence and redaction server-side when privacy is at stake; hidden CSS is not a privacy assertion.
- Separate cold-start, compile, server-readiness, and business-journey assertions so infrastructure defects do not masquerade as product failures.
- Do not overspecify animation frames, durations, or DOM clones unless the governing presentation contract requires them.
- Snapshot tests need bounded, reviewed semantics and may not replace behavior assertions for critical paths.

### 8.3 Assertion weakening review

| Change                | Required justification                                                                              |
|-----------------------|-----------------------------------------------------------------------------------------------------|
| Remove assertion      | Behavior retired or stronger accepted assertion covers same risk.                                   |
| Increase timeout      | Measured environment or performance evidence, diagnosed cause, new budget, and no race concealment. |
| Add retry             | Eligible failure class, fresh runtime, preserved first failure, flake policy.                       |
| Relax selector/state  | Old assertion proved incidental detail; replacement proves equal or stronger semantic contract.     |
| Skip platform/browser | Capability unavailable or unsupported under explicit compatibility policy and release effect.       |

## PART III

## 9. Test-Review Checklist

Test review is a first-class engineering review, not a rubber stamp after production code has already been approved. Reviewers evaluate whether the proposed tests protect the contract, fail for the intended reason, remain maintainable, and operate safely under the Sounding Line runtime.

### 9.1 Required review questions

| Area             | Questions                                                                                                          |
|------------------|--------------------------------------------------------------------------------------------------------------------|
| Intent           | What behavior or risk does this test protect? Would it fail if the defect returned?                                |
| Authority        | Does it assert the canonical service, event, projection, version, or user outcome rather than a retired mirror?    |
| Negative paths   | Are invalid, unauthorized, stale, duplicate, provider-failure, and recovery states covered or explained?           |
| Privacy/security | Could fixture data, logs, traces, screenshots, URLs, or error messages expose secrets or private content?          |
| Accessibility    | Can the behavior be completed and understood with keyboard, zoom, reduced motion, no audio, and declared fallback? |
| Isolation        | Does the suite own its ports, data, accounts, storage, processes, and provider namespace? Is ordering hidden?      |
| Timing           | Are waits event-based and budgets justified? Are sleeps, retries, and timeouts masking a race?                     |
| Metadata         | Are suite ID, contracts, owners, resources, risk, expected duration, and release gates correct?                    |
| Evidence         | Will the executor produce normalized results, artifacts, and cleanup receipts required by the gate?                |

### 9.2 Review outcomes

| Outcome                 | Meaning                                                                                                              |
|-------------------------|----------------------------------------------------------------------------------------------------------------------|
| APPROVED                | Test and metadata meet the contract and runtime policy.                                                              |
| APPROVED_WITH_FOLLOW_UP | Non-blocking improvement is recorded with owner and deadline; no hidden risk reduction.                              |
| CHANGES_REQUIRED        | Assertion, negative coverage, fixture, ownership, metadata, isolation, or evidence is insufficient.                  |
| POLICY_BLOCKED          | The proposed test weakens a mandatory contract, leaks data, uses unsafe resources, or relies on prohibited behavior. |
| NEEDS_CONTRACT_OWNER    | Product meaning or accepted replacement contract is genuinely ambiguous.                                             |

### 9.3 Reviewer separation

- Critical security, privacy, migration, data-loss, backup/restore, and release infrastructure changes require review by an owner independent from the implementing agent where practical.
- Codex may prepare evidence and self-audit, but it does not grant its own policy exception or release GO.
- Infrastructure owners review resource and cleanup safety; product owners review semantic assertions; gate owners review release sufficiency.
- A single person may hold several roles in a small project, but the receipt must record which authority made each decision.

#### Review principle

A test is accepted because it protects a contract safely and reproducibly, not because it makes the pull request green. Green is an outcome; correctness is the reason.



## PART III

# 10. Release-Gate Classes

Release gates define distinct validation boundaries. A lower gate may prove a smaller scope quickly; it may not promote itself into a broader status. Every gate declares its required suites, contracts, environments, freshness, resource and cleanup requirements, permissible external deferrals, and decision authority.

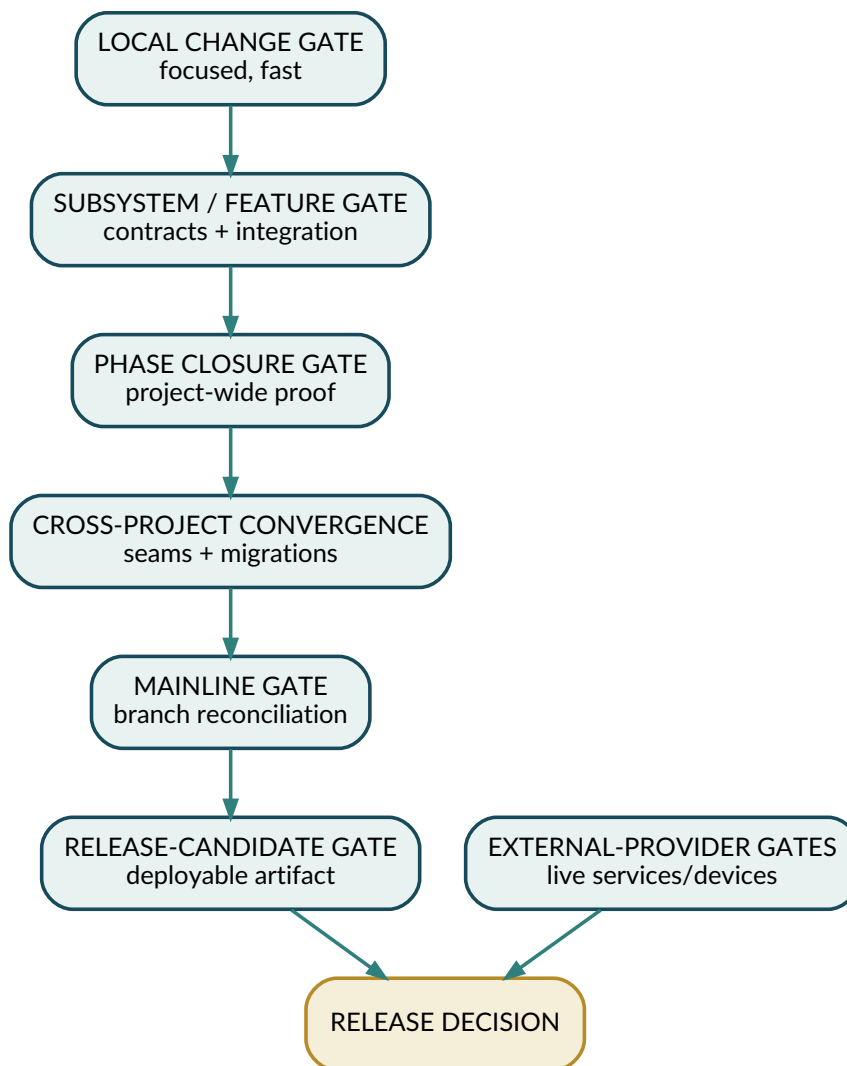


Figure 4. Sounding Line gate ladder from a focused local change through release-candidate and external-provider proof.

## 10.1 Gate catalog

| Gate              | Purpose                                                | Minimum evidence                                                                                 | Typical decision                    |
|-------------------|--------------------------------------------------------|--------------------------------------------------------------------------------------------------|-------------------------------------|
| LOCAL_CHANGE      | Fast developer/Codex feedback for changed contracts.   | Impact-selected Tier 0-3 and required focused browser nodes; exact source and cleanup.           | FOCUSED_VALIDATED or NOT_VALIDATED. |
| SUBSYSTEM_FEATURE | Prove one feature or subsystem and adjacent contracts. | All affected tiers, negative paths, integration seams, migrations if changed.                    | INTEGRATION_VALIDATED.              |
| PHASE_CLOSURE     | Close a governed project phase.                        | Phase checklist, full project families, browser/accessibility, build/migrations, receipts, gaps. | Phase completion classification.    |

| Gate                      | Purpose                                                                            | Minimum evidence                                                                                                 | Typical decision                             |
|---------------------------|------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|----------------------------------------------|
| CROSS_PROJECT_CONVERGENCE | Prove combined branches and shared ownership boundaries.                           | Merged migration order, contract suites, canonical data protection, browser journeys, build/restart as required. | INTEGRATION_VALIDATED candidate.             |
| MAINLINE                  | Authorize merge to canonical main.                                                 | Fresh origin/main reconciliation, required convergence gate, policy checks, clean resources, parity.             | Mainline merge allowed or denied.            |
| RELEASE_CANDIDATE         | Prove a deployable immutable artifact and supported environment.                   | Tier 7 release plan, production build, migrations, browsers, accessibility, security/privacy, rollback, cleanup. | RELEASE_GO / NO_GO / INCOMPLETE.             |
| EXTERNAL_PROVIDER         | Prove real MySQL, object storage, scanner, KMS, systemd, device, or alerting path. | Real service identity, isolated namespace, success and failure, cleanup, truthful status.                        | Live-validated, external-pending, or failed. |

## 10.2 Gate policy invariants

- Every gate is versioned and references registered suites and contracts; ad hoc command lists are transitional only.
- A gate failure does not erase passing lower-tier evidence, but it prevents promotion beyond the proven status.
- A mainline or release gate may reuse fresh valid evidence only when source, policy, environment, dependency, and gate rules permit it.
- External-provider gates remain separate so unavailable credentials do not hide incomplete local implementation or fabricate live proof.
- Gate duration targets guide infrastructure optimization; they never authorize removal of mandatory evidence.
- A gate decision is invalid if required cleanup, evidence integrity, or count reconciliation is missing.

## 10.3 Gate ownership

The gate owner decides sufficiency under Parts I-III. Product and contract owners define required semantic coverage; the Part II runtime proves execution and cleanup; release management applies the final release decision. Codex and ordinary developers may request and report gates but may not self-authorize exceptions outside their assigned authority.

## PART III

# 11. Mainline Policy

The mainline policy protects the canonical repository from stale bases, unproven migrations, incomplete integration, ambiguous evidence, and abandoned resources. Main is not merely the place where branches eventually go. It is the authoritative source state against which later work, releases, and historical evidence are interpreted.

## 11.1 Mandatory pre-merge state

| Requirement           | Policy                                                                                                               |
|-----------------------|----------------------------------------------------------------------------------------------------------------------|
| Remote truth          | Fetch origin and record current origin/main. The branch base and merge-base must be known.                           |
| Branch reconciliation | Inspect mainline advances affecting source, contracts, migrations, policy, dependencies, fixtures, or release gates. |
| Gate evidence         | Required subsystem, phase, or convergence gates passed against the actual candidate source.                          |
| Migration order       | SQLite/MySQL identifiers, checksums, dependencies, empty and upgrade rehearsals are coherent.                        |
| Evidence freshness    | No mandatory result invalidated by subsequent source, policy, dependency, fixture, or environment change.            |
| Repository state      | Targeted staged diff, no unrelated files, no secrets, no test databases, no unowned generated artifacts.             |
| Runtime state         | Task-owned processes, ports, databases, roots, leases, and locks are released or formally transferred.               |
| Privacy/security      | Repository and staged-diff privacy scans clean; sensitive artifacts retained only in approved evidence roots.        |
| Parity                | Final commit pushed; remote branch parity and intended ancestry verified.                                            |

## 11.2 When main advances

- Do not reset the candidate to an old remembered SHA or overwrite newer accepted work.
- Determine whether evidence remains valid by changed contracts, source dependencies, schemas, fixtures, package lock, policy version, and environment.
- Create a new plan for invalidated evidence; rerun only nodes whose proof is no longer valid plus gate-required descendants.
- Resolve conflicts by current product ownership and governing documents, not by mechanically preferring either branch.
- If migration identity or shared contract ownership collides, stop before mutating canonical data and obtain an explicit integration decision.
- Record the actual base, candidate, merge commit, and post-merge validation source identity.

## 11.3 Post-merge requirements

| Action                                                     | Evidence                                                                        |
|------------------------------------------------------------|---------------------------------------------------------------------------------|
| Verify main ancestry and remote parity                     | Commit IDs, merge-base, ahead/behind zero where expected.                       |
| Run required post-merge smoke or gate                      | Plan and receipt tied to merged main, not only pre-merge branches.              |
| Confirm canonical data unchanged or intentionally migrated | Checksums, row counts, schema version, migration receipt.                       |
| Release resources                                          | Cleanup receipt, port/process absence, worktree status.                         |
| Update status and feature records                          | Ledgerlight/project status/Feature Catalog links and completion classification. |

### Mainline hard boundary

A branch that was release-ready yesterday is not automatically safe to merge today after main changed. Evidence is reusable only when its declared identity and invalidation rules still hold.

## PART III

# 12. Release Decision Policy

Release decisions use exact statuses and authorities. They are derived from mandatory gate evidence, not from test totals, confidence, urgency, or a completion message that has become emotionally invested in shipping. A decision must be reproducible from the source, policy, plan, environment, evidence, exceptions, and cleanup receipts.

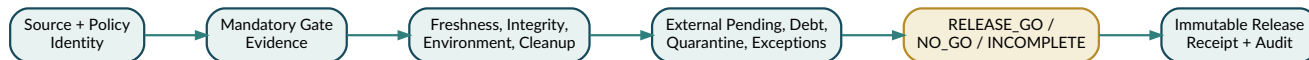


Figure 5. Release decision flow from source identity and mandatory evidence through exceptions, final status, and immutable audit.

## 12.1 Release decision statuses

| Status                           | Meaning                                                                                                                                                      | Permitted conditions                                                                                             |
|----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| RELEASE_GO                       | All mandatory local, integration, compatibility, migration, security/privacy, build, rollback, and required external gates passed with valid fresh evidence. | No unresolved release-blocking failure, debt, quarantine, invalid evidence, or cleanup issue.                    |
| RELEASE_GO_WITH_EXTERNAL_PENDING | Locally release-ready and specifically authorized external proof remains pending.                                                                            | Only gate-defined external dependencies; exact residual risk, environment restriction, owner, and follow-up.     |
| RELEASE_NO_GO                    | One or more required behaviors or gates failed.                                                                                                              | Independent product, test, infrastructure, security, migration, cleanup, or provider failure remains unresolved. |
| RELEASE_INCOMPLETE               | Required evidence did not run, is blocked, stale, cancelled, or missing.                                                                                     | No claim that product behavior failed; release cannot proceed.                                                   |
| EVIDENCE_INVALID                 | Receipts cannot be trusted or do not match source, policy, environment, plan, integrity, or cleanup rules.                                                   | Gate must be rerun or incident resolved.                                                                         |

## 12.2 Decision authority

| Authority                        | May do                                                                                 | May not do                                                                                                               |
|----------------------------------|----------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| Developer/Codex                  | Request gates, produce implementation and evidence reports, identify external pending. | Grant own exception, quarantine, mainline approval, or release GO unless explicitly assigned and independently governed. |
| Test/contract owner              | Approve semantic coverage and classify test/contract defects.                          | Override release policy or evidence integrity.                                                                           |
| Infrastructure owner             | Approve runtime, isolation, cleanup, and external-provider evidence.                   | Redefine product success.                                                                                                |
| Gate owner/integrator            | Evaluate gate completeness and convergence.                                            | Convert missing evidence into pass.                                                                                      |
| Release manager/authorized owner | Issue release decision from valid gate bundle and approved exceptions.                 | Ignore mandatory failures or invalid evidence.                                                                           |

## 12.3 Required decision wording

Release status: RELEASE\_GO | RELEASE\_GO\_WITH\_EXTERNAL\_PENDING | RELEASE\_NO\_GO | RELEASE\_INCOMPLETE | EVIDENCE\_INVALID  
Source identity: <commit / artifact digest>  
Policy and gate version: <IDs>  
Mandatory evidence: <complete / failed / missing>  
External gates: <status and scope>  
Quarantine / debt / exceptions: <IDs and release effect>  
Cleanup and canonical-state proof: <receipt IDs>  
Approver and timestamp: <authority>

### No percentage releases

“98% complete” may be useful planning language. It is not a release status. One unresolved authorization defect is not transformed into safety by dividing it among 967 passing tests.

## PART III

# 13. Flake Policy

A flake is a diagnosed instability pattern that can produce different outcomes under equivalent declared inputs. It is not any test that failed once and passed after retry. Flake classification preserves first-pass evidence, requires an owner and expiry, and never permits indefinite retry until green.

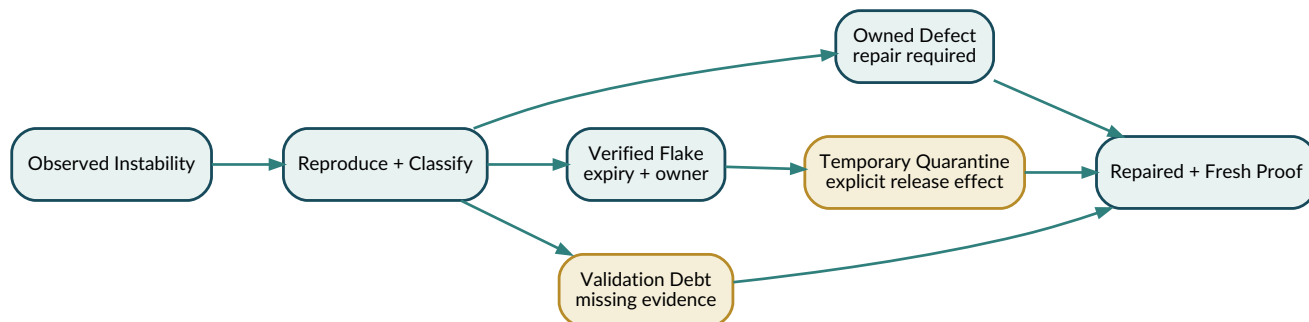


Figure 6. Governed path from observed instability through classification, defect repair, flake/quarantine, validation debt, and fresh proof.

## 13.1 Flake qualification

| Requirement             | Evidence                                                                                                         |
|-------------------------|------------------------------------------------------------------------------------------------------------------|
| Equivalent inputs       | Same source, policy, suite version, fixture, environment class, resource profile, and intended provider outcome. |
| Repeated observation    | Multiple attempts or historical signatures showing instability rather than one unexplained failure.              |
| Bounded fingerprint     | Sanitized failure signature, phase, location, timing, and affected resources.                                    |
| Owner and cause class   | Product, test, fixture, infrastructure, dependency, environment, or unknown owner.                               |
| Risk and release effect | Critical contracts may block release despite intermittent pass.                                                  |
| Remediation and expiry  | Investigation plan, deadline, maximum attempts, and quarantine/debt relationship if used.                        |

## 13.2 Retry rules

- Retries are permitted only for registered eligible classes and maximum attempt counts.
- Every retry uses a fresh runtime or explicitly documented retained state appropriate to the failure.
- The original failure, artifacts, resource state, and timing remain in the receipt.
- A later pass is reported as `PASSED_AFTER_RETRY`, not a clean pass.
- Security, authorization, privacy, migration, data-loss, backup/restore, progression idempotency, and evidence-integrity failures are not casually reclassified as flakes.
- A high retry rate triggers slow-suite/flake review even when the gate eventually passes.

## 13.3 Flake closure

Closure requires a diagnosed repair, fresh deterministic or stress evidence, historical signature absence over the governed observation window, metadata update, and removal of any related quarantine or debt. Merely reaching the expiry date does not cure the test.

### Flake truth rule

Retry is a diagnostic tool, not a laundering service for red test results. A green second attempt does not retroactively make the first attempt imaginary.

## PART III

# 14. Quarantine Policy

Quarantine is a temporary, visible, governed state for a test that cannot currently participate normally without producing misleading or unsafe decisions. It is not a trash bin for inconvenient tests. The default quarantine registry is empty, and every entry has authority, evidence, owner, expiry, remediation, and release impact.

## 14.1 Required quarantine record

| Field                 | Requirement                                                                                     |
|-----------------------|-------------------------------------------------------------------------------------------------|
| Quarantine ID         | Stable machine-readable identifier.                                                             |
| Suite/case/contract   | Exact affected identity and version.                                                            |
| Reason/classification | Diagnosed product, test, fixture, infrastructure, environment, or compatibility issue.          |
| Evidence              | Failure signatures, reproduction, attempts, and why ordinary execution is unsafe or misleading. |
| Risk                  | Affected behavior, contract criticality, negative paths, and user impact.                       |
| Owner and approver    | Responsible repair owner and authorized quarantine authority.                                   |
| Start and expiry      | Time-bounded period; no indefinite entries.                                                     |
| Release behavior      | Block, conditional restriction, alternate evidence, or non-release-only omission.               |
| Compensating controls | Replacement suite, manual proof, environment restriction, or additional monitoring.             |
| Exit criteria         | Repair and fresh evidence required to remove entry.                                             |

## 14.2 Prohibited quarantine uses

- Quarantining a test because it slows a release or exposes a real defect.
- Quarantining an authorization, privacy, migration, data-loss, backup/restore, or idempotency test without exceptional release authority.
- Using quarantine to avoid updating stale routes, fixtures, or terminology after an intentional architecture change.
- Hiding the entry from plans, reports, pull requests, project status, or release receipts.
- Automatically renewing expiry without review and new evidence.
- Counting a quarantined case as passed or subtracting it silently from planned totals.

## 14.3 Plan and gate behavior

The planner includes quarantine metadata and reason. Depending on the gate, the node may execute as non-authoritative diagnostic work, be skipped by explicit policy, require compensating evidence, or block the decision. The gate receipt reconciles the case separately from failed, blocked, cancelled, and not-run counts.

### Default state

Quarantine is an emergency shelf with a date label, not the attic. If the registry grows comfortably, governance has failed.

## PART III

# 15. Validation-Debt Policy

Validation debt records required evidence that is known to be missing. Debt is not a euphemism for unfinished implementation or a silent promise that someone will remember later. It has an owner, risk, deadline, future gate, workaround, and explicit release effect, and it appears in project status until closed.

## 15.1 Debt record schema

| Field                       | Required content                                                                                                                       |
|-----------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| Debt ID and status          | OPEN, MITIGATED, BLOCKED_EXTERNAL, OVERDUE, CLOSED.                                                                                    |
| Missing capability/evidence | Exact suite, contract, provider, browser, device, environment, or historical fixture.                                                  |
| Affected behavior and risk  | What remains unproven and worst credible consequence.                                                                                  |
| Cause                       | External credential, infrastructure absence, tool defect, time-bounded migration, historical incompatibility, or other classification. |
| Owner and escalation        | Person/project responsible and authority for overdue or release-blocking review.                                                       |
| Compensating evidence       | Local simulation, manual review, restricted rollout, monitoring, or alternate provider proof.                                          |
| Deadline and required gate  | Date/event when the debt must be resolved and exact future gate.                                                                       |
| Release effect              | Block, external-pending restriction, feature flag, environment limitation, or informational only.                                      |
| Closure evidence            | Fresh plan/receipt, commit, provider identity, cleanup, and status update.                                                             |

## 15.2 Debt boundaries

- Locally attainable implementation and tests must be completed before classifying a live provider or device gate as external debt.
- Debt cannot be used to defer core authorization, privacy, data integrity, migration safety, or rollback without explicit release authority.
- Overdue debt automatically escalates risk and may invalidate conditional release authorization.
- Debt follows the affected contract across branches and releases; merging does not close it.
- Debt records are machine-readable and linked from project status, completion receipts, and release decisions.
- Closing debt requires evidence, not a status edit.

## 15.3 External-pending example

A real AWS KMS exercise may remain BLOCKED\_EXTERNAL after the adapter, encryption-context policy, fake-client contract tests, key rotation logic, failure tests, and local provider proof are complete. The debt identifies the managed key, isolated environment requirement, exact live operations, owner, and release restriction. It does not claim KMS is live-validated because a mock returned a cheerful object.



## PART III

# 16. Evidence-Retention Policy

Evidence retention balances reproducibility, incident investigation, privacy, storage cost, and the fact that many test artifacts are far more revealing than their file extensions suggest. Retention is determined by outcome, risk, gate, privacy class, and legal or project policy. Deletion must preserve required integrity manifests and release decisions.

## 16.1 Retention classes

| Evidence class      | Default retention intent                                                                   | Examples                                                             |
|---------------------|--------------------------------------------------------------------------------------------|----------------------------------------------------------------------|
| TRANSIENT_CLEAN     | Short-lived; retain summary and hashes, delete bulky ordinary artifacts after gate window. | Unit stdout, successful browser video, disposable DB clone.          |
| FOCUSED_PROOF       | Retain through branch/phase review and evidence freshness window.                          | Focused receipts, screenshots, migration rehearsal, build fragments. |
| FAILURE_DIAGNOSTIC  | Retain until defect resolution plus review window; privacy-scrubbed.                       | Trace, process tree, failed DB clone, logs, network events.          |
| FLAKE_INVESTIGATION | Retain across observation window and closure.                                              | Multiple attempts, timing/resource history, signatures.              |
| SECURITY_OR_PRIVACY | Restricted access, encrypted or protected storage, incident-controlled retention.          | Exploit reproduction, redaction failure, sensitive traces.           |
| RELEASE_EVIDENCE    | Long-lived immutable receipt and required artifacts tied to released build.                | Gate decision, manifests, migration/build/cleanup proofs.            |
| EXTERNAL_PROVIDER   | Retain provider identity, safe commands/results, isolated namespace and cleanup evidence.  | MySQL, S3, ClamAV, KMS, Linux service proof.                         |
| INCIDENT            | Retain under incident policy until authorized closure and governance review.               | Worktree overwrite, canonical DB mutation, evidence tampering.       |

## 16.2 Integrity and access

- Evidence manifests contain hashes, source/policy/plan/run identity, retention class, privacy classification, and storage location.
- Artifacts are immutable after sealing; later annotations are separate signed or versioned records.
- Release and incident evidence has access controls and tamper detection appropriate to its sensitivity.
- Private Chronicle prose, accepted answers, credentials, real personal media, exact private locations, and raw provider secrets are excluded or redacted by default.
- Deletion is logged with authority, manifest, reason, and retained decision metadata.
- A missing required artifact can invalidate evidence even when the original command exited successfully.

## 16.3 Evidence minimization

Retain what is necessary to reproduce the decision, diagnose defects, prove cleanup, and audit release assurance. Do not retain enormous videos, raw databases, or browser storage merely because the runtime produced them. Conversely, do not delete the only evidence of a security, migration, or cleanup failure because disk space became socially awkward.

## PART III

# 17. Security and Privacy Policy for Tests

Tests must protect private data and system credentials with the same seriousness as production code. Synthetic fixtures are the default. The existence of an isolated runtime does not authorize copying production users, private Chronicle content, personal media, real invitation secrets, or provider credentials into test artifacts.

## 17.1 Test security requirements

| Area                    | Policy                                                                                                                                             |
|-------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| Synthetic identities    | Use reserved example domains, deterministic display names, scoped roles, and no real personal information.                                         |
| Credentials and secrets | Use task-owned test secrets or provider secret references; never commit, log, screenshot, or embed them in URLs.                                   |
| Cookies/storage state   | Run-owned, access-controlled, redacted from ordinary logs, destroyed at cleanup unless required evidence.                                          |
| Private media/content   | Use synthetic images/audio/video/text with approved classification; protected fixtures remain outside public repository paths.                     |
| Locations               | Use synthetic coordinates or approved public sites; exact private homes or personal routes are prohibited.                                         |
| Malicious fixtures      | EICAR, path traversal, malformed archives, hostile metadata, and similar fixtures are clearly marked, isolated, and excluded from public delivery. |
| Logs/traces/screenshots | Redaction, bounded payloads, no accepted answers, PINs, tokens, object keys, private paths, or unpublished prose.                                  |
| Supply chain            | Locked dependencies, trusted scripts/runners, no arbitrary community package execution, and package-script policy.                                 |
| Authorization           | Foreign-account and wrong-role tests use distinct identities and verify safe not-found/redacted outputs.                                           |

## 17.2 Repository scanning

- Run repository and staged-diff privacy/secret scans for changes touching fixtures, traces, screenshots, private content, environment configuration, or generated outputs.
- Test artifact directories are ignored from Git but also actively scanned; Gitignore is not a security boundary.
- Detected possible private content stops replication and triggers review; automated history rewriting is prohibited without separate authorization.
- Public fixtures document provenance and license where binary/media assets are included.
- Policy validators reject NEXT\_PUBLIC or client-visible configuration for secrets and private provider material.

## 17.3 Incident triggers

Real personal data, credentials, private Chronicle prose, exact private location, unredacted access token, or protected media in a public test artifact is a security/privacy incident. Stop admission, preserve bounded evidence, revoke exposed credentials where applicable, classify scope, and follow Section 28. Passing tests do not reduce the seriousness of the leak.

### Security boundary

A test does not become safe because it runs locally, and a fixture does not become synthetic because someone renamed the file sample.json.

## PART III

# 18. Test-Data Governance

Test data is a governed product asset. Fixtures have owners, provenance, schema versions, privacy classifications, compatibility expectations, retention, and deletion rules. The platform must be able to explain who created a fixture, what contract it represents, which suites consume it, and whether it may appear in a public repository or evidence artifact.

## 18.1 Fixture classes

| Class                  | Purpose                                                                                       | Default handling                                            |
|------------------------|-----------------------------------------------------------------------------------------------|-------------------------------------------------------------|
| PUBLIC_SYNTHETIC       | Generic safe state suitable for committed tests and examples.                                 | Repository allowed; no personal/private material.           |
| PROTECTED_SYNTHETIC    | Security, private-media, malformed package, or restricted scenario fixture.                   | Protected test root; bounded access and artifacts.          |
| GENERATED_EPHEMERAL    | Run-created identities, invitations, databases, objects, tokens, and browser state.           | Run-owned; cleanup mandatory; summaries retained.           |
| FROZEN_HISTORICAL      | Accepted prior schema/version/output needed for compatibility.                                | Immutable, checksummed, provenance and reader expectations. |
| EXTERNAL_PROVIDER_TEST | Synthetic records placed into isolated real provider namespaces.                              | Provider-specific retention and cleanup evidence.           |
| INCIDENT_REPRODUCTION  | Restricted evidence reproducing an incident when public synthetic substitute is insufficient. | Incident-controlled; never ordinary fixture.                |

## 18.2 Fixture governance fields

- Stable fixture ID, owner, schema version, creation source, privacy class, license/provenance, contracts, consumers, and checksum.
- Allowed environments and whether the fixture may be committed, uploaded, screenshotted, or shared publicly.
- Determinism rules, random seed, time base, identity namespace, expected row/object counts, and cleanup behavior.
- Migration/upcasting policy and reader compatibility for frozen historical fixtures.
- Retirement criteria and replacement mapping when product authority changes.
- Health tests that detect schema drift, missing assets, invalid relationships, duplicate identifiers, and accidental secrets.

## 18.3 Production data prohibition

Production data is not copied into ordinary development or CI testing. Where a real-world scale or distribution is needed, create synthetic generators or approved anonymized research datasets under a separate privacy process. Anonymization must be justified against re-identification risk; deleting a display name while preserving email, timestamps, coordinates, and prose is not anonymization.

## 18.4 Consent and provenance

Any non-synthetic asset or dataset used for testing requires documented rights, consent or legal basis, purpose, retention, access, and deletion. The default Chronicles repository test corpus remains synthetic because platform verification should not depend on somebody donating their relationship history to a browser matrix.

## PART III

# 19. Performance Budgets

Performance budgets ensure that rigorous verification remains usable as the repository grows. Budgets measure planning, queueing, provisioning, setup, execution, teardown, and total wall time separately so optimization targets the actual bottleneck. A missed budget creates an owned performance obligation; it never authorizes weaker assertions or omitted evidence.

## 19.1 Initial target budgets

| Scope                        | Target wall time    | Notes                                                                                          |
|------------------------------|---------------------|------------------------------------------------------------------------------------------------|
| Plan generation              | < 10 seconds        | Ordinary source diff and policy registry; conservative expansion remains deterministic.        |
| Focused static               | < 60 seconds        | Formatting, TypeScript slice, lint/architecture/language/privacy as impacted.                  |
| Impacted unit/component      | < 3 minutes         | Typical non-migration change on supported workstation profile.                                 |
| Impacted service integration | < 8 minutes         | Includes isolated database/provider setup where required.                                      |
| Focused browser family       | < 10 minutes        | Warm business journey; cold-start proof is separate.                                           |
| Subsystem closure            | < 25 minutes        | Complete affected subsystem and contract evidence.                                             |
| Cross-project/phase closure  | < 45 minutes target | May exceed for recovery, lifecycle, or large migration gates; critical path must be explained. |
| Full local release gate      | < 60 minutes target | Excludes genuinely external provider wait; includes cleanup.                                   |
| Distributed CI release gate  | < 30 minutes target | Requires sufficient trusted worker capacity and deterministic sharding.                        |

## 19.2 Budget decomposition

| Metric                | Meaning                                                                   | Owner response                                           |
|-----------------------|---------------------------------------------------------------------------|----------------------------------------------------------|
| queue_seconds         | Time ready work waits for resource or capacity.                           | Resource capacity, priority, fairness, contention.       |
| provision_seconds     | Runtime root, leases, database, browser, service, or provider allocation. | Part II adapter/baseline owner.                          |
| setup_seconds         | Fixture, migration, warmup, compile, generation, or build preparation.    | Suite/infrastructure owner; reuse analysis.              |
| execution_seconds     | Actual executor work.                                                     | Test owner; split, algorithm, fixture, assertion review. |
| teardown_seconds      | Process/resource release and cleanup verification.                        | Resource adapter owner.                                  |
| critical_path_seconds | Longest dependency path determining wall time.                            | Planner/scheduler and owners on path.                    |
| total_wall_seconds    | User-observed duration through terminal cleanup.                          | Gate owner and platform capacity.                        |

## 19.3 Budget governance

- Budgets are defined per capacity profile, gate, environment, and cold/warm mode; comparisons use equivalent conditions.
- Percentiles and trends are preferred over one unusually fast or slow run.
- A budget miss records the top queue, setup, execution, and teardown contributors plus historical comparison.
- Optimizations require evidence that selection, contracts, negative coverage, compatibility, and cleanup remain intact.
- Hardware pressure, thermal throttling, and concurrent user activity are recorded rather than blamed vaguely on the workstation.
- Budget changes require policy review when they redefine release expectations or hide sustained regression.

### Budget rule

A one-hour release gate that provides trustworthy evidence may be a performance problem. A ten-minute gate that quietly skips the difficult half is a correctness problem. Sounding Line fixes the first without becoming the second.

## PART III

## 20. Slow-Suite Governance

Slow suites receive explicit governance before they dominate the critical path and become untouchable rituals. Duration alone does not make a suite bad; some recovery, lifecycle, browser, and external-provider proofs are inherently expensive. The policy requires owners to explain the cost, verify value, and pursue safe decomposition, reuse, or concurrency.

### 20.1 Review triggers

| Trigger                                                   | Required review                                                                         |
|-----------------------------------------------------------|-----------------------------------------------------------------------------------------|
| Duration exceeds gate threshold or 95th percentile budget | Critical-path contribution, setup/execution split, historical trend.                    |
| Growth > 20% across governed observation window           | Source/test/fixture/runtime change causing regression.                                  |
| Suite is top critical-path node repeatedly                | Split, shard, parallel-safety, fixture reuse, alternate tier placement.                 |
| Retries or timeouts contribute materially                 | Flake diagnosis, event-based waits, resource pressure, provider health.                 |
| High setup-to-execution ratio                             | Certified baseline, shared immutable setup, warmup separation, adapter caching.         |
| Large case count with uneven shards                       | Historical balancing, case isolation, long-tail family extraction.                      |
| Global-exclusive classification                           | Revalidate smallest correct exclusive resource; prove why narrower lease is impossible. |

### 20.2 Slow-suite record

- Suite ID/version, owner, contracts, risk, gate membership, environment and capacity profile.
- Median, p90, p95, p99, queue, setup, execution, teardown, and critical-path contribution.
- Why the evidence belongs at the current tier and gate.
- Candidate optimizations: split, shard, baseline reuse, fixture simplification, read-only server, parallel lane, algorithm or assertion improvement.
- Rejected optimizations and the coverage or correctness they would weaken.
- Target date, expected savings, acceptance test, and rollback.

### 20.3 Safe optimization order

1. Remove accidental waits, duplicate setup, and repeated compilation.
2. Reuse certified immutable baselines and safe shared read-only services.
3. Split unrelated test families and isolate long-tail cases.
4. Shard by historical duration after parallel safety is proven.
5. Reduce fixture size only when the protected boundary remains representative.
6. Improve production/test algorithms and diagnostics.
7. Change mandatory coverage or gate placement only through governing amendment.

#### No invisible two-hour rituals

A suite may remain slow because it proves something expensive and important. It may not become slow by historical neglect while everyone quietly schedules lunch around it.

## PART III

## 21. Historical Result Governance

Historical verification results improve planning, sharding, budget analysis, flake investigation, and risk review. They are not current proof. Historical data is bounded, sanitized, source-aware, and prohibited from becoming production-user analytics or a machine-learning oracle that overrides declared contracts.

### 21.1 Governed historical uses

| Use                  | Allowed inputs                                                                     | Prohibited conclusion                                         |
|----------------------|------------------------------------------------------------------------------------|---------------------------------------------------------------|
| Duration estimation  | Suite version, environment, capacity profile, attempt class, duration percentiles. | A historically fast suite may be omitted.                     |
| Shard balancing      | Case/family durations, setup cost, resource profile.                               | Cases may share unsafe mutable state for balance.             |
| Flake detection      | Sanitized signatures, retry patterns, environment, resource pressure.              | A real regression is merely flaky because it occurred before. |
| Impact review        | Past contract co-failures and source families as advisory signals.                 | Historical correlation replaces ownership/contract graph.     |
| Capacity planning    | Queue/resource wait, worker capabilities, browser/database slots.                  | More concurrency is safe without isolation proof.             |
| Regression detection | Budget trends, cleanup failures, evidence invalid rates.                           | A single comparison proves a performance regression.          |
| Risk correlation     | Recurring defect families and critical-path incidents.                             | Risk is reduced without governing amendment.                  |

### 21.2 Data minimization and retention

- No production-user activity, private Chronicle content, accepted answers, personal media, credentials, raw request payloads, or exact private locations.
- Store normalized repository-relative source families, suite/contract IDs, bounded fingerprints, environment and resource classes.
- Retain release/incident history longer than ordinary local noise; delete raw bulky artifacts according to Section 16.
- Access to security/privacy incident signatures is restricted and audited.
- Historical records include policy and suite version so obsolete data is not compared as if semantics were unchanged.

### 21.3 Governance review

Quarterly or milestone review should identify suites with increasing duration, retry rate, cleanup failures, queue contention, evidence invalidity, and recurrent root signatures. Owners propose correction, resource changes, or policy updates. Historical metrics may prioritize work but cannot waive a mandatory gate or manufacture confidence from yesterday's pass.

#### Historical evidence boundary

History tells the scheduler what usually happens. The current run proves what happened now.

## PART III

## 22. Documentation Requirements

Sounding Line documentation must keep policy, implementation, operations, and evidence connected without duplicating every concept into six contradictory files. Documentation is versioned, owned, linked from machine-readable policy, and updated in the same task when behavior or enforcement changes.

### 22.1 Required documentation families

| Document family            | Required content                                                                                                | Owner                             |
|----------------------------|-----------------------------------------------------------------------------------------------------------------|-----------------------------------|
| Governing documents        | Parts I-III, amendments, precedence, hard boundaries, final rules.                                              | Program governance.               |
| Architecture/ADRs          | Planner, runtime, adapters, policy validators, gate decisions, technology choices, rollback.                    | Architecture owner.               |
| Suite and contract catalog | Human-readable purpose, owners, tiers, resources, risks, gates, examples.                                       | Test/contract owners.             |
| Runbooks                   | Local execution, durable follow/cancel/resume, external providers, cleanup, orphan recovery, incident handling. | Infrastructure/operations owners. |
| Gate definitions           | Required suites/contracts/environments/evidence, status, authority, external deferrals.                         | Gate owner.                       |
| Performance history        | Budgets, critical paths, slow-suite reviews, capacity profiles.                                                 | Infrastructure and suite owners.  |
| Status/receipts            | Implementation phase, latest release proof, debt, quarantine, exceptions, blockers, provider status.            | Project/release owners.           |
| Training/templates         | Developer, Codex, integrator, release manager, maintainer workflows and examples.                               | Program governance.               |

### 22.2 Repository locations

```

Development_Docs/
  Project_Sounding_Line_*.pdf
  Project_Sounding_Line_*_Design_Record.md
  Project_Sounding_Line_*_Validation_Record.md
  Project_Sounding_Line_Completion_Receipt.md

docs/testing/
  architecture.md
  local-development.md
  ci-and-release.md
  failure-and-incident-guide.md
  external-provider-validation.md

testing/
  *.json policy files
  schemas/
  templates/
  fixtures/ metadata

```

### 22.3 Documentation truth rules

- Do not write completion or live-provider claims before the evidence exists.
- Historical reports remain intact; later phases add closure or correction notes rather than rewriting an improbably perfect past.
- Examples use synthetic identifiers, safe paths, and current terminology.
- Generated catalogs declare source policy version and generation timestamp.
- Broken links, stale command names, unknown suite IDs, and contradictory statuses are policy-validation failures.
- A PDF diagram may explain architecture, but machine-readable files remain enforcement inputs.

## PART III

## 23. Machine-Readable Policy Files

Machine-readable policy files allow the repository to validate ownership, contracts, impact, resources, gates, quarantine, debt, capacities, and exceptions instead of relying on prompt wording or human memory. Files are schema-versioned, deterministic, reviewable, and generated projections never replace their canonical source.

### 23.1 Canonical policy files

| File                           | Authority                                                                                                              |
|--------------------------------|------------------------------------------------------------------------------------------------------------------------|
| testing/suites.json            | Suite identity, version, tier, owner, command/executor, contracts, paths, resources, duration, parallel status, gates. |
| testing/contracts.json         | Contract identity, owner, producers/consumers, success/failure semantics, versions, required tiers, risk.              |
| testing/ownership.json         | Source, test, contract, infrastructure, gate, escalation owners and aliases.                                           |
| testing/resources.json         | Resource classes, capacities, compatibility, sharing/exclusivity, adapter identity, cleanup requirements.              |
| testing/impact-map.json        | Direct paths, imports, contracts, schema/routes, escalation rules, uncertainty handling.                               |
| testing/release-gates.json     | Gate versions, mandatory suites/contracts/evidence/environments, external policy, decision authority.                  |
| testing/quarantine.json        | Time-bounded quarantine entries and release effects.                                                                   |
| testing/validation-debt.json   | Missing evidence, risk, owner, deadline, future gate, release effect.                                                  |
| testing/exceptions.json        | Approved temporary policy deviations, scope, expiry, compensating controls.                                            |
| testing/policies.json          | Cross-cutting retry, timeout, retention, privacy, assertion, review, and completion policies.                          |
| testing/capacity-profiles.json | Workstation/CI worker resources, concurrency, reserves, disk and pressure thresholds.                                  |

### 23.2 Versioning and review

- Each file declares schema version, policy version, canonical ordering, and content digest.
- References use stable IDs, not display names or paths that may change.
- A policy change that affects selection or decision invalidates plans and evidence according to Part I rules.
- Human-readable generated docs are reviewed with the canonical JSON diff.
- High-risk changes to release gates, risk floors, privacy, retries, quarantine, or exceptions require designated owner review and often an ADR.
- Unknown fields fail closed unless the schema defines an extension namespace.

### 23.3 No prompt-only policy

Major test obligations may be repeated in a Codex prompt for clarity, but their authority comes from the repository policy and governing documents. A prompt must not silently narrow the plan, invent a release gate, or grant a permanent exception that cannot be audited after the conversation ends.



## PART III

## 24. Policy Validation

The policy validator is a Tier 0 release-critical tool. It rejects inconsistent, incomplete, cyclic, unowned, unsafe, or expired policy before plan generation. A repository whose test policy cannot validate itself cannot claim deterministic selection or release assurance.

### 24.1 Required validation checks

| Category                  | Checks                                                                                                                           |
|---------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| Identity                  | Unique suite, contract, owner, gate, resource, debt, exception, and quarantine IDs; valid versions and canonical ordering.       |
| References                | All owners, contracts, paths, resources, dependencies, gates, fixtures, and policies resolve.                                    |
| Graph                     | No suite dependency cycles, missing nodes, impossible cleanup chain, or release gate without producer path.                      |
| Ownership                 | Every production and critical test family has source/test/contract/infrastructure/gate ownership or explicit unresolved blocker. |
| Risk/tier                 | Risk classes use valid floors; critical contracts cannot map only to shallow or quarantined evidence.                            |
| Resources                 | Known resource classes, valid capacities, compatible bundles, parallel-safety evidence and revocation state.                     |
| Quarantine/debt/exception | Approver, owner, reason, expiry, release effect, compensating controls; no overdue silent entry.                                 |
| Impact                    | Paths and contracts point to existing sources/suites; unknown-impact behavior is conservative.                                   |
| Gate completeness         | Mandatory suites/contracts/evidence/environments and decision authority exist.                                                   |
| Privacy/security          | No secrets, unsafe paths, production identifiers, or client-visible private configuration.                                       |

### 24.2 Validator outputs

```
Policy validation: PASS | FAIL
Policy version: <version>
Schema versions: <file -> version>
Files validated: <count>
Errors: <stable codes and exact paths>
Warnings: <owned and reviewable>
Expired entries: <quarantine/debt/exceptions>
Unresolved ownership: <IDs>
Digest: sha256:<...>
```

### 24.3 Change enforcement

- Policy validation runs on local affected plans, pull requests, mainline, and release gates.
- The validator itself has unit, schema, negative, migration, and compatibility tests.
- A change to validator behavior requires frozen policy fixtures and differential proof against the prior accepted version.
- Warnings cannot conceal release-blocking errors; categories and severity are governed.
- Auto-fix is permitted only for deterministic formatting or clearly reversible metadata normalization, never for ownership or risk decisions.

#### Policy fail-closed rule

A missing owner, broken contract reference, or expired exception does not make the repository simpler. It makes the required proof unknowable, so closure stops until the policy is repaired.

## PART III

## 25. Repository Status Integration

Sounding Line status integrates with project control-plane records so verification state is visible beside implementation state. A project cannot be represented as simply COMPLETE when its current source, release gate, external providers, debt, quarantine, or evidence freshness tell a more complicated story.

### 25.1 Required status fields

| Field                | Meaning                                                                                |
|----------------------|----------------------------------------------------------------------------------------|
| governingBaseline    | Accepted Sounding Line and project document versions.                                  |
| implementationPhase  | Take the Soundings, Open the Channels, Read the Current, Prove the Passage, or closed. |
| policyVersion        | Current validated policy digest and schemas.                                           |
| runtimeVersion       | Part II orchestrator/broker/executor implementation identity.                          |
| lastFocusedProof     | Latest relevant focused receipt by source/contract.                                    |
| lastIntegrationProof | Latest cross-project or subsystem proof and freshness.                                 |
| lastReleaseProof     | Latest Tier 7 source/artifact, environment, decision, and cleanup.                     |
| externalProviders    | Implemented/configured/live-validated/blocked-external/failed per provider.            |
| validationDebt       | Open/overdue debt IDs and release effects.                                             |
| quarantines          | Active entries, expiry, owner, and gate impact.                                        |
| exceptions           | Active temporary deviations and expiry.                                                |
| performanceBudgets   | Current gate budgets, latest critical path, regressions.                               |
| activeBlockers       | Product, test, infrastructure, policy, provider, incident, or evidence blocker.        |

### 25.2 Integration targets

- Ledgerlight consumes verification and gate status without becoming the source of test truth.
- Feature Catalog links features to critical contracts, owning suites, and latest release evidence.
- Project status files show implementation and validation separately.
- Completion receipts reference immutable Sounding Line receipts rather than copying unstructured console text.
- Release records include gate decision, external-pending scope, policy/runtime versions, artifact digest, and rollback identity.
- Dashboards label stale, incomplete, blocked, and external evidence honestly.

### 25.3 Status reconciliation

A periodic validator checks status records against policy, evidence store, branches, and expiry dates. It detects claimed release validation without a valid receipt, closed debt without closure proof, expired quarantine, missing external gate, and project completion based only on an unmerged branch. Status is a projection of evidence, not an editable optimism field.

## PART III

## 26. Codex Prompt Contract

Every major Codex implementation prompt must include a consistent test and repository contract. Project-specific prompts define product scope; this section defines the minimum execution behavior so an agent cannot interpret caution as permission to stop after one batch or interpret autonomy as permission to damage shared work.

### 26.1 Mandatory prompt clauses

| Clause                 | Required direction                                                                                                |
|------------------------|-------------------------------------------------------------------------------------------------------------------|
| Repository truth       | Fetch/verify current source; historical SHA is evidence, not restoration command.                                 |
| Branch/worktree safety | Task-owned branch/worktree, preserve unrelated changes, no destructive Git, targeted staging.                     |
| Ownership and scope    | Read governing documents, identify owners/contracts/non-goals/shared files.                                       |
| Test obligations       | Source and tests same task, negative cases, metadata/impact updates, focused then expanded evidence.              |
| Runtime policy         | Use Sounding Line durable run, task-owned resources, no duplicate runs, follow/cancel/resume existing run.        |
| Continuity             | Ordinary setup and failures are agent responsibility; continue independent work when one gate is blocked.         |
| No silent weakening    | No assertion deletion, arbitrary sleeps, unconditional skips, broad retries, timeout inflation without diagnosis. |
| Completion             | Report exact statuses, evidence, omissions, debt, external gates, cleanup, commit, parity; no interim completion. |
| Privacy/security       | Synthetic data, scans, secret handling, no private content or production data.                                    |
| Authority              | Do not merge/release/grant exception unless prompt and repository authority explicitly allow it.                  |

### 26.2 Reusable prompt skeleton

#### Repository and source truth:

- Fetch current origin/main and record actual base.
- Preserve unrelated work; use task-owned worktree and resources.

#### Behavior and contracts:

- State changed behavior, owners, affected contracts, risk, non-goals.
- Update source, tests, metadata, fixtures, and impact rules in the same task.

#### Execution:

- Generate the Sounding Line plan.
- Run focused proof, then required integration/compatibility/release nodes.
- Continue independent work during external or shared-gate waits.

#### Completion:

- Do not stop while locally attainable mandatory scope remains.
- Report evidence-bound status, omissions, debt, external gates, cleanup, commit, and parity.

### 26.3 Prompt precedence

The prompt may be more specific than repository policy but may not silently weaken it. If a prompt requests destructive Git, unowned resources, omitted critical tests, or an unsupported release claim, Codex must refuse that instruction and preserve the higher governing authority. If a newer accepted governing amendment changes policy, the prompt is reconciled to the amendment and the conflict is recorded.

### Codex governance rule

A long prompt is not good governance if its structure teaches the agent that every checkpoint is an honorable place to quit. Product scope may be staged; completion behavior must still require continuous work through all locally attainable mandatory scope.

## PART III

## 27. Codex Completion Report Format

Codex completion reports use a stable structure so implementation, validation, release, and external-provider status cannot blur into one triumphant paragraph. The report is a human-readable projection of machine-readable policy and evidence receipts, with exact source and cleanup identity.

### 27.1 Required completion format

```
Implementation status: IMPLEMENTED | FOCUSED_VALIDATED | INTEGRATION_VALIDATED | COMPATIBILITY_VALIDATED |
RELEASE_VALIDATED | EXTERNAL_VALIDATION_PENDING | BLOCKED | NOT_VALIDATED
```

```
Behavior changed:
Contracts affected:
Owners and boundaries:
Files changed:
Tests added:
Tests updated:
Policy/metadata/fixtures updated:
```

```
Focused evidence:
Integration evidence:
Compatibility evidence:
Release evidence:
External-provider evidence:
```

```
Omitted or not-run suites and reasons:
First-pass failures / retries / flakes:
Quarantine / validation debt / exceptions:
Cleanup and canonical-state proof:
Known limitations:
```

```
Branch / commit:
Remote parity:
Evidence receipt IDs:
Next authorized action:
```

### 27.2 Wording rules

| Wording                     | Use                                                                                    |
|-----------------------------|----------------------------------------------------------------------------------------|
| IMPLEMENTED                 | Production-intended code exists; required validation not yet complete.                 |
| FOCUSED_VALIDATED           | Impact-selected relevant suites passed for declared scope.                             |
| INTEGRATION_VALIDATED       | Affected cross-project/service/browser seams passed.                                   |
| COMPATIBILITY_VALIDATED     | Required browser/device/database/provider compatibility passed.                        |
| RELEASE_VALIDATED           | Full governed release gate passed for exact artifact/source.                           |
| EXTERNAL_VALIDATION_PENDING | All locally attainable work complete; named live provider/device proof remains.        |
| BLOCKED                     | A genuine dependency prevents further required work and independent work is exhausted. |
| NOT_VALIDATED               | Implementation exists without sufficient evidence or evidence is invalid/stale.        |

### 27.3 Prohibited ambiguity

- Do not use “complete” without the qualifying validation status and merge/release state.
- Do not count simulated or fake-provider proof as live-validated.
- Do not present a busy shared validation lock as a blocker to unrelated implementation, tests, documentation, or browser work.
- Do not hide retries, cleanup failures, synthetic data spills, or unmerged status in footnotes.
- Do not claim mainline or remote parity without verifying it.
- Do not include raw secrets, private prose, invitation values, object keys, or sensitive artifact paths in the report.

### Completion report rule

The report must make the most cautious truthful reading easy. It should not require detective work to discover that “finished” means “the branch exists, except the release gate, merge, external provider, and cleanup are still pending.”

## PART III

## 28. Incident Policy

Incidents are events in which the verification system, repository, canonical data, credentials, evidence, or release declaration may no longer be trustworthy. Incident response prioritizes containment, evidence preservation, ownership, and safe recovery over finishing the current test run or preserving an optimistic status.



Figure 7. Sounding Line incident lifecycle from detection through containment, recovery, authorized resumption, and governance review.

### 28.1 Incident classes

| Class                           | Examples                                                                                         | Initial severity |
|---------------------------------|--------------------------------------------------------------------------------------------------|------------------|
| REPOSITORY_DAMAGE               | Worktree overwrite, destructive cleanup, rewritten accepted history, corrupted generated source. | Critical         |
| CANONICAL_DATA_MUTATION         | Shared/canonical database changed by test, migration, browser, or fixture.                       | Critical         |
| PRIVATE_DATA_OR_SECRET_EXPOSURE | Credential, private Chronicle prose/media, exact location, invitation secret, unredacted trace.  | Critical         |
| EVIDENCE_INTEGRITY              | Tampered/missing receipt, source mismatch, forged cleanup, invalid release decision.             | High to Critical |
| RESOURCE_OWNERSHIP              | Killed unrelated process, port hijack, cross-run database/account/storage contamination.         | High             |
| BASELINE_CORRUPTION             | Certified database/build/fixture baseline modified or misidentified.                             | High             |
| ORPHANED_RUNTIME                | Unowned controller, processes, locks, provider namespaces, or databases survive.                 | Moderate to High |
| INCORRECT_RELEASE_DECLARATION   | Release GO or completion status issued without mandatory evidence.                               | High             |
| POLICY_BYPASS                   | Critical test skipped, quarantine hidden, exception expired, validator disabled.                 | High             |

### 28.2 Immediate response

- Stop new admission to affected runs, gates, resources, or releases.
- Preserve bounded logs, state journals, manifests, process/resource identity, source, policy, and artifacts without copying private content unnecessarily.
- Contain only resources whose ownership is proven; do not perform broad cleanup that could destroy evidence or unrelated work.
- Revoke exposed credentials, signed URLs, sessions, keys, or provider access according to owning project policy.
- Record incident ID, reporter, time, affected source/environment, known scope, and initial classification.
- Notify repository, security/privacy, infrastructure, product, and release owners according to severity.

### 28.3 Recovery and resumption

| Stage                  | Requirement                                                                                                     |
|------------------------|-----------------------------------------------------------------------------------------------------------------|
| Scope confirmation     | Determine affected branches, worktrees, databases, artifacts, releases, providers, and evidence.                |
| Integrity verification | Compare canonical source, hashes, migration state, baselines, backups, provider objects, and release records.   |
| Recovery               | Restore from accepted source/backup, remove only proven owned artifacts, rotate secrets, repair policy/runtime. |
| Evidence revocation    | Invalidate any receipt or release decision whose source, environment, resources, or cleanup cannot be trusted.  |

| Stage                | Requirement                                                                                             |
|----------------------|---------------------------------------------------------------------------------------------------------|
| Fresh proof          | Generate a new plan and rerun mandatory affected gates after recovery.                                  |
| Resumption authority | Authorized owner records why normal work and releases may resume.                                       |
| Post-incident review | Root cause, contributing conditions, missed detections, tests, policy/ADR changes, owner and deadlines. |

**Stop-work rule**  
When canonical source, data, credentials, or evidence may be compromised, continuing to collect more green test results is not progress. It is contamination.



## PART III

## 29. Exception Policy

Exceptions permit a temporary, explicit, bounded deviation when a governing requirement cannot be satisfied immediately and the residual risk is knowingly accepted by the correct authority. Exceptions do not silently change policy, erase failures, or become permanent through neglect.

### 29.1 Required exception request

| Field                 | Requirement                                                                                      |
|-----------------------|--------------------------------------------------------------------------------------------------|
| Exception ID          | Stable identifier and policy clauses affected.                                                   |
| Requestor and owner   | Person/project seeking deviation and owner responsible for closure.                              |
| Reason                | Concrete technical or external constraint; convenience alone is insufficient.                    |
| Scope                 | Exact source, branch, gate, suite, contract, environment, release, and duration.                 |
| Risk analysis         | Behavior unproven, credible consequence, affected users/data, reversibility.                     |
| Compensating controls | Alternate evidence, restricted rollout, feature flag, monitoring, manual verification, rollback. |
| Approver              | Authority independent from the requesting agent when the risk is material.                       |
| Expiry and revocation | Date/event, automatic invalidation conditions, and overdue behavior.                             |
| Release effect        | Allowed status and explicit restrictions.                                                        |
| Closure               | Required fresh evidence, cleanup, status update, and removal from registry.                      |

### 29.2 Non-exceptional requirements

- An exception cannot classify failed or invalid evidence as passed.
- An exception cannot authorize destructive Git, unowned cleanup, secret exposure, production data in tests, or hidden quarantine/debt.
- Authorization, privacy, data-loss, migration, backup/restore, evidence integrity, and release-signing requirements need exceptional authority and generally block public release.
- A missing local setup step, uninstalled dependency, absent disposable database, or ordinary test failure is not an exception case.
- External-provider absence is handled by explicit external-pending/debt policy before exception policy.
- Expired exceptions fail closed and may invalidate conditional release status.

### 29.3 Exception review

The approver verifies that all locally attainable work is complete, the exception is the narrowest possible deviation, compensating controls are real, the rollback path is available, and the release wording exposes the restriction. Codex may draft an exception request but may not approve its own deviation.

#### Exception rule

An exception is a temporary bridge over a known gap, not a trapdoor beneath the governing documents.

## PART III

## 30. Governance Changes Requiring ADR

Material governance changes require an Architecture Decision Record so the repository preserves why the decision changed, which alternatives were rejected, how compatibility is maintained, and what evidence proves the new rule. Ordinary metadata edits do not need an ADR; changes to meaning, authority, or risk do.

### 30.1 ADR-required changes

| Change                                                      | Why ADR is required                                        |
|-------------------------------------------------------------|------------------------------------------------------------|
| New or redefined test tier                                  | Changes evidence depth and selection semantics.            |
| New resource class or ownership model                       | Changes isolation, security, cleanup, or concurrency.      |
| New/changed release gate                                    | Changes what is required to merge or release.              |
| Risk-floor reduction or mandatory coverage reduction        | Directly weakens proof and requires explicit authority.    |
| Changed failure/status/evidence semantics                   | Affects historical interpretation and release confidence.  |
| Changed quarantine, flake, debt, exception, or retry policy | Changes how unresolved evidence affects releases.          |
| Changed Codex obligations or completion authority           | Affects autonomous implementation and reporting.           |
| Changed privacy/security or test-data policy                | Could expose data or alter threat controls.                |
| Legacy harness retirement/cutover                           | Changes authoritative execution and rollback.              |
| Distributed worker trust or external-provider boundary      | Changes supply-chain, secret, and environment assumptions. |

### 30.2 ADR structure

ADR ID and title  
 Status and effective policy version  
 Context and problem  
 Governing clauses affected  
 Decision and authority  
 Alternatives considered  
 Security/privacy/data implications  
 Compatibility and migration  
 Implementation phases  
 Validation and acceptance evidence  
 Rollback and emergency mode  
 Documentation/policy file updates  
 Consequences and unresolved risks

### 30.3 Amendment relationship

- An ADR may implement or interpret existing policy within its boundaries.
- If the decision changes a normative Part I-III clause, a governing amendment is also required.
- The policy validator references accepted ADRs where machine-readable behavior depends on them.
- Superseded ADRs remain in history and point to replacements.
- Emergency decisions receive a provisional incident/exception record and a retrospective ADR before permanent adoption.

## PART III

# 31. Implementation Roadmap Detail

Project Sounding Line is implemented in four phases. Part III governs what developers, Codex, CI, integrators, and release owners are required to do during each phase. The phases reduce risk in sequence: understand current verification, isolate execution, enforce intelligent selection and evidence, then make the system authoritative for release.

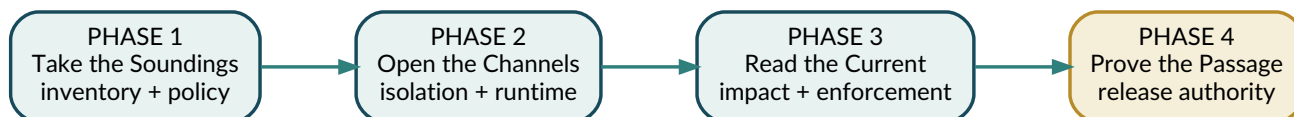


Figure 8. Four-phase Sounding Line governance roadmap: inventory, isolation, impact enforcement, and release authority.

## 31.1 Phase 1 - Take the Soundings

| Element             | Governance requirement                                                                                               |
|---------------------|----------------------------------------------------------------------------------------------------------------------|
| Prerequisites       | Accepted Parts I-III baseline; repository access; current harness inventory.                                         |
| Deliverables        | Suite/contract/owner/resource/gate inventory, policy schemas, status language, plan-only output, legacy command map. |
| Developer/Codex use | Declare behavior and tests; metadata may be incomplete but gaps are visible and conservative.                        |
| Prohibited claims   | No broad parallel speedup, impact precision, or authoritative new release decision yet.                              |
| Acceptance          | Every current suite is found or explicitly unknown; critical owners/contracts/gates identified; policy validates.    |
| Rollback            | Metadata and plan-only tooling can be disabled without changing current harness execution.                           |

## 31.2 Phase 2 - Open the Channels

| Element             | Governance requirement                                                                                        |
|---------------------|---------------------------------------------------------------------------------------------------------------|
| Prerequisites       | Phase 1 policy stable enough to declare resources and dependencies.                                           |
| Deliverables        | Part II broker/runtime, isolated roots, databases, ports, servers, browsers, cleanup, parallel certification. |
| Developer/Codex use | Task-owned runs and resources mandatory for migrated suites; legacy lock narrowed.                            |
| Prohibited claims   | No impact-selected omission or authoritative release cutover before comparison evidence.                      |
| Acceptance          | Independent suites run concurrently without contamination; cleanup and orphan recovery proven.                |
| Rollback            | Emergency serial/legacy wrapper remains available.                                                            |

## 31.3 Phase 3 - Read the Current

| Element             | Governance requirement                                                                                                            |
|---------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Prerequisites       | Reliable isolated execution and historical result capture.                                                                        |
| Deliverables        | Impact planner, root/cascade normalization, evidence reuse/invalidation, Codex enforcement, stale-test and slow-suite governance. |
| Developer/Codex use | Plan generation required; change-to-test and completion report contracts enforced.                                                |
| Prohibited claims   | Impact selection may not replace full release gates.                                                                              |
| Acceptance          | Plans explain selection/omission; failures collapse correctly; local and CI affected decisions align.                             |
| Rollback            | Full explicit gate remains available; planner can broaden to conservative all-relevant mode.                                      |

## 31.4 Phase 4 - Prove the Passage

| Element       | Governance requirement                                               |
|---------------|----------------------------------------------------------------------|
| Prerequisites | Phases 1-3 accepted; dual-run evidence; security and privacy review. |

| Element             | Governance requirement                                                                                                   |
|---------------------|--------------------------------------------------------------------------------------------------------------------------|
| Deliverables        | Trusted CI/distributed execution, authoritative release receipts, branch protection, cutover, audit, legacy deprecation. |
| Developer/Codex use | Sounding Line gates become required for mainline/release; completion language validated.                                 |
| Prohibited claims   | Legacy and new systems may not issue conflicting authoritative release decisions.                                        |
| Acceptance          | Release decisions reproducible; local/CI parity; performance budgets measured; rollback tested; closure receipt.         |
| Rollback            | Emergency serial mode and legacy compatibility wrapper retained through observation window.                              |

## 31.5 Phase governance records

- Each phase creates design, implementation, validation, integration, performance, security/privacy, and completion records appropriate to scope.
- Phase completion uses Part I validation language and does not imply later phases exist.
- Defects found during later phases add correction notes to earlier records rather than rewriting history.
- No phase may consume later authority simply because the implementation code happens to exist.

## PART III

## 32. Adoption Plan

Adoption is incremental. Existing tests are not rewritten wholesale before the architecture can run them. Each suite moves through controlled stages, preserving current coverage while gaining metadata, wrappers, isolation, parallel certification, impact selection, and eventual retirement of obsolete harness assumptions.

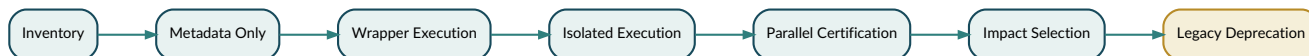


Figure 9. Seven-stage adoption path from inventory to safe legacy-harness deprecation.

### 32.1 Migration stages

| Stage                     | Repository behavior                                                                                   | Exit criteria                                                       |
|---------------------------|-------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|
| 1. Inventory              | Suite discovered, command/path/cases/duration/resources/owners recorded; current execution unchanged. | No hidden current suite family; unknowns explicit.                  |
| 2. Metadata-only          | Stable suite ID, contracts, risk, resources, gate membership, parallel status UNCLASSIFIED.           | Policy validates and planner can explain selection.                 |
| 3. Wrapper execution      | Existing command invoked through Sounding Line adapter with source/run/evidence identity.             | Results/counts match legacy; cleanup recorded.                      |
| 4. Isolated execution     | Run-owned database/ports/storage/accounts/processes; canonical state protected.                       | Repeated success/failure/cancel cleanup; no contamination.          |
| 5. Parallel certification | Concurrent collision tests and resource classification.                                               | Declared safety status accepted or constrained.                     |
| 6. Impact selection       | Contracts/paths/risk drive affected plan; evidence invalidation and rerun supported.                  | Selection explainable and conservative; comparison accepted.        |
| 7. Legacy deprecation     | Old command becomes wrapper or is retired after observation and rollback proof.                       | No unique coverage/authority remains; documentation and CI updated. |

### 32.2 No big-bang requirement

- Suites may migrate by project, tier, resource family, or gate based on value and risk.
- Legacy commands remain usable as compatibility wrappers while policy and evidence become authoritative.
- Unmigrated suites remain selected explicitly for gates and are not treated as parallel safe.
- High-risk and high-contention suites receive early migration priority.
- Migration must preserve case count, semantics, artifacts, environment, and cleanup or document approved differences.
- The adoption dashboard distinguishes inventory, wrapper, isolation, certification, selection, and retirement status.

### 32.3 Prioritization

| Priority | Candidate suites                                                                                                     |
|----------|----------------------------------------------------------------------------------------------------------------------|
| First    | Global-lock browser families, shared SQLite/MySQL, fixed ports, long lifecycle loops, migration/build/restart gates. |
| Second   | High-risk authorization/privacy/progression/private-content and cross-project contract suites.                       |
| Third    | Large unit/component families that benefit from metadata and sharding.                                               |
| Later    | Stable fast pure suites with little contention, while still receiving metadata.                                      |

#### Adoption rule

Do not postpone all value until every test has been rewritten. Also do not call the program complete because three demonstration suites run in parallel while the release gate still waits behind the old global lock.

## PART III

## 33. Training and Usage Guidance

Sounding Line changes how several roles work. Training must teach the practical workflow, the meaning of statuses, how to inspect durable runs, how to avoid duplicate work, and when to escalate. Documentation and examples are role-specific so people are not forced to learn the entire orchestration engine before fixing a button.

### 33.1 Ordinary developer workflow

- State intended behavior and request a plan for changed files/contracts.
- Run focused evidence locally; follow the durable run rather than launching duplicate commands.
- Add/update tests, fixtures, metadata, and negative cases in the same change.
- Inspect root failures and blocked descendants; repair the root before retrying broadly.
- Use the pull-request template to report evidence, omissions, debt, and cleanup.
- Do not claim release validation from a local affected plan.

### 33.2 Codex workflow

- Read governing and project context; verify repository/worktree/source state.
- Generate expected plan and risk before broad editing.
- Continue autonomously through ordinary setup, focused repair, testing, documentation, commit, and push.
- Use task-owned runtime resources and durable status/follow/cancel/resume operations.
- Do not weaken tests, use destructive Git, touch other worktrees, or declare ordinary inconvenience a blocker.
- Return the governed completion report with exact status and evidence.

### 33.3 Integrator workflow

- Fetch all candidate branches and current origin/main; inspect ownership, migration ranges, shared policy, and evidence freshness.
- Build the convergence plan and execute cross-project seams with isolated resources.
- Resolve stale tests by preserving risk, not deleting journeys.
- Reconcile root/cascade failures, migration order, build/restart, privacy, cleanup, and remote parity.
- Issue integration status; do not grant release status unless authorized gate passes.

### 33.4 Release manager workflow

- Select exact source/artifact and release-gate policy version.
- Verify mandatory evidence, external gates, debt/quarantine/exceptions, rollback, cleanup, and integrity.
- Issue one of the exact release statuses with restrictions and authority.
- Retain release receipt and monitor external-pending closure or conditional restrictions.
- Revoke decision if evidence becomes invalid or incident scope reaches the release.

### 33.5 Test infrastructure maintainer workflow

- Maintain planner, broker, executors, adapters, policy validator, baselines, workers, evidence and cleanup.
- Review contention, duration, orphan recovery, cleanup failures, and evidence integrity.
- Certify or revoke parallel safety and capacity profiles.
- Support incident recovery without redefining product semantics.
- Publish migration and compatibility records for runtime changes.

### 33.6 Training assets

| Asset             | Content                                                                          |
|-------------------|----------------------------------------------------------------------------------|
| Quick-start guide | Plan, run, status, follow, cancel, evidence, common failures.                    |
| Role playbooks    | Developer, Codex, integrator, release manager, maintainer steps and authority.   |
| Failure examples  | Root vs cascade, flake, stale test, invalid evidence, cleanup incident.          |
| Policy exercises  | Classify change type, choose negative cases, evaluate exception/debt/quarantine. |

| Asset                 | Content                                                                             |
|-----------------------|-------------------------------------------------------------------------------------|
| Reference commands    | Local, browser, subsystem, release, external provider, recovery.                    |
| Onboarding validation | Small practical exercise proving the user can execute and interpret a governed run. |

## PART III

## 34. Final Program Acceptance Criteria

Project Sounding Line is complete only when its architecture, runtime, repository enforcement, and release assurance operate together across the real repository. Document completion alone is not program completion. Demonstration parallelism alone is not program completion. The final gate proves that the repository knows what must be tested, executes it safely, enforces obligations, and produces trustworthy release decisions.

### 34.1 Final program acceptance criteria

| Domain               | Acceptance                                                                                                                                                        |
|----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Policy inventory     | Every active suite has valid metadata, owner, tier, contracts, resources, risk, parallel status, duration, and gate membership or an explicit unresolved blocker. |
| Critical contracts   | Every critical contract has required positive, negative, lifecycle, authorization/privacy, persistence, compatibility, and release coverage.                      |
| Impact planning      | Representative changes produce deterministic, explainable, conservative plans; unknowns broaden rather than disappear.                                            |
| Parallel runtime     | Independent suites run concurrently using isolated resources; global exclusivity is rare, justified, and tested.                                                  |
| Canonical protection | Canonical databases, worktrees, storage, ports, accounts, and providers remain unchanged unless explicitly governed.                                              |
| Durability/recovery  | Runs survive client exit, worker/controller crash, cancellation, stale lease, and machine restart with truthful evidence and cleanup.                             |
| Failure truth        | Root failures and cascade-blocked nodes reconcile; retry/flake/quarantine/debt/exception states remain distinct.                                                  |
| Codex enforcement    | Major prompts and completions follow the contract; tests and source evolve together; prohibited weakening and destructive behavior are detected.                  |
| Local/CI parity      | Equivalent policy and source produce compatible plan and decision locally and in trusted CI/distributed workers.                                                  |
| Release assurance    | Mainline and release gates issue reproducible decisions with source/artifact, policy, evidence, external status, cleanup, and authority.                          |
| Performance          | Budgets and critical paths are measured; focused and release targets are met or owned without weakening proof.                                                    |
| Security/privacy     | Policy, fixtures, evidence, workers, providers, and cleanup pass threat-model and incident scenarios.                                                             |
| Adoption             | Legacy harness is retired or retained only as a tested compatibility wrapper with no competing release authority.                                                 |
| Documentation/status | Parts I-III, policy files, runbooks, status, debt/quarantine/exception registers, and completion receipt agree.                                                   |

### 34.2 Final closure evidence

- Full Tier 7 release gate executed through Sounding Line against an immutable release candidate.
- Dual-run and cutover records prove acceptable equivalence and improved diagnostic/resource behavior.
- Security/privacy review, policy audit, incident drills, external-provider classifications, and rollback exercise complete.
- Performance report covers plan, queue, provisioning, setup, execution, teardown, critical path, and capacity profiles.
- Canonical repository/data/resource checks and cleanup receipt are clean.
- Remote mainline and release artifact parity verified.
- Completion ledger contains zero unowned critical suites/contracts, zero expired hidden entries, and no ambiguous release authority.
- Final Project Sounding Line completion receipt is immutable, linked from project status, and names the observation/rollback period.



## 34.3 Final program status

PROJECT SOUNDING LINE  
PROGRAM STATUS: COMPLETE

Part I - Software Verification Architecture: ACCEPTED  
Part II - Execution Infrastructure and Parallel Runtime: ACCEPTED  
Part III - Repository Policy, Codex Governance, and Release Assurance: ACCEPTED

Authoritative release gates: SOUNDING LINE  
Legacy harness: RETIRED or COMPATIBILITY WRAPPER ONLY  
Open release-blocking debt: 0  
Expired quarantines/exceptions: 0  
Canonical data/resource contamination: 0  
Final release proof: VALID  
Rollback/emergency serial mode: PROVEN

### Final governing acceptance

Sounding Line is finished when rigorous verification becomes ordinary repository behavior: planned before execution, isolated before parallelism, enforced during change, truthful during failure, and reproducible at release. The program does not exist to make the terminal greener. It exists to make the green result mean something.

## APPENDIX A

# Appendix A. Normative Change-to-Test Matrix

This matrix is a normative starting point for policy files and prompt templates. The planner may expand beyond the listed floors. “Not applicable” requires a reason tied to the changed contract.

| Change                    | Tier floor                    | Required negative/failure evidence                                                          | Typical escalations                               |
|---------------------------|-------------------------------|---------------------------------------------------------------------------------------------|---------------------------------------------------|
| Pure domain/schema parser | Tier 1                        | Invalid type, unknown field, boundaries, canonicalization, version.                         | Historical fixtures, consumer contract.           |
| React component behavior  | Tier 2                        | Loading, error, disabled, keyboard, focus, reduced motion if relevant.                      | Tier 4 journey, viewport/zoom.                    |
| Navigation/shell          | Tier 2 + 4                    | Unauthorized role, back/forward, focus restoration, mobile menu, active Voyage restriction. | Tier 5 True North/project seams, Tier 6 browsers. |
| API read                  | Tier 1 + 3                    | Unauthorized, foreign account, invalid query/cursor, redaction, not-found enumeration.      | Public projection/search/cache.                   |
| API mutation              | Tier 1 + 3                    | CSRF, validation, wrong role, stale revision, duplicate, idempotency, partial failure.      | Tier 4 browser and audit.                         |
| Prisma/schema/migration   | Tier 0 + 3                    | Empty/upgrade, drift, FK/integrity, rollback/restore, both engines.                         | Affected service/browser, release gate.           |
| Session/auth              | Tier 1 + 3 + 4                | Expired/revoked, cross-role, replay, rate limit, CSRF, rotation, session fixation.          | Tier 5 identity/project seams, Tier 6 browser.    |
| Privacy/public projection | Tier 1 + 3                    | IDOR, safe not-found, field allowlist, logs/traces/OG/search exclusion.                     | Tier 4 public route, privacy scan.                |
| Animation/lifecycle       | Tier 1 + 2 + 4                | Missing targets, remount, interruption, replay, fallback, reduced motion, cleanup.          | Tier 6 browser/viewport/audio.                    |
| Private content/package   | Tier 1 + 3                    | Tamper, wrong key, path traversal, scanner unavailable, quarantine, rollback, cleanup.      | Provider/recovery/browser gate.                   |
| Geolocation/map/provider  | Tier 1 + 3                    | Denied, stale, low accuracy, impossible jump, offline, manual fallback, privacy.            | Real device/provider, browser compatibility.      |
| Build/dependency          | Tier 0 + build                | Clean install, generated drift, package scripts, supply-chain, rollback.                    | Production browser/restart.                       |
| Test infrastructure       | Tier 0 + contract/integration | Lease collision, orphan, cancellation, cleanup, evidence tampering, rollback.               | Full Part II acceptance.                          |
| Policy/gate change        | Tier 0 + differential         | Invalid references, expiry, conflicting statuses, prior-policy fixtures.                    | ADR, dual-run, mainline/release.                  |

## Matrix use

The row identifies the minimum floor for the change itself. Contracts, risk, affected consumers, historical compatibility, and requested gate may add nodes. No matrix row authorizes omission of a known transitive risk.

## APPENDIX B

# Appendix B. Codex Preflight and Completion Checklists

---

## B.1 Codex preflight checklist

- Current repository root, branch, commit, upstream, origin/main, merge base, worktrees, staged/unstaged/untracked state recorded.
- Accepted governing documents and project status identified; historical SHAs treated as evidence only.
- Task-owned branch/worktree/resources selected; active shared owners and migration ranges inspected.
- Behavior statement, non-goals, owners, contracts, risk, change class, negative categories, and expected gate stated.
- Current source/tests/fixtures/metadata read; stale assumptions and compatibility surfaces identified.
- Expected Sounding Line plan generated or manually mapped during transition.
- Ordinary setup path known: locked dependencies, isolated database, ports, runtime root, evidence root.
- No destructive Git, broad staging, production/private data, duplicate durable run, or other-worktree access planned.

## B.2 Codex implementation checklist

- Production behavior and tests updated in same task.
- Semantic assertions preserved or strengthened.
- Required negative cases implemented or explicitly non-applicable.
- Policy/metadata/fixtures/impact rules updated.
- Task-owned resources and durable run used.
- Focused reproducer and affected family run before heavyweight gate.
- First-pass failures diagnosed; no arbitrary sleeps/timeouts/retries/skips.
- Independent work continued while one external/shared gate waited.
- Privacy/security scans and artifact handling appropriate to change.

## B.3 Codex completion checklist

- No substantial locally attainable mandatory scope remains.
- Changed files and behaviors reconciled.
- Contracts/owners and tests added/updated listed.
- Focused/integration/compatibility/release/external evidence separated.
- Omitted/not-run nodes and reasons listed.
- Retries/flakes/quarantine/debt/exceptions disclosed.
- Cleanup and canonical-state proof complete.
- Commit/push/parity verified where required.
- Validation status uses exact vocabulary.
- Next action is authorized and does not silently begin another phase.

### Completion veto

Any unchecked item that affects the requested gate prevents a completion claim. The task may still report IMPLEMENTED or BLOCKED with exact evidence, but it may not improvise a stronger status.

## APPENDIX C

# Appendix C. Gate Evidence Matrix

The gate matrix provides a concise view of evidence families. Actual gate definitions in `testing/release-gates.json` are authoritative and may contain project-specific requirements.

| Evidence family         | Local              | Subsystem           | Phase                    | Convergence       | Mainline          | Release candidate         | External                       |
|-------------------------|--------------------|---------------------|--------------------------|-------------------|-------------------|---------------------------|--------------------------------|
| Policy validation       | Required           | Required            | Required                 | Required          | Required          | Required                  | Required                       |
| Impacted Tier 0-3       | Required           | Required            | Required                 | Required          | Required          | Required                  | As applicable                  |
| Focused browser         | If journey         | Required if surface | Required                 | Required          | Required          | Required                  | Provider UI if any             |
| Cross-project contracts | If affected        | Required if seam    | Required                 | Required          | Required          | Required                  | Provider boundary              |
| Compatibility matrix    | If affected        | Risk-based          | Project-defined          | Affected matrix   | Required subset   | Full supported matrix     | Real environment               |
| SQLite empty/upgrade    | If schema          | If schema           | Required if project uses | Required          | Required          | Required                  | N/A                            |
| MySQL schema/live       | If schema          | Prepared/risk       | Project-defined          | Required prepared | Required prepared | Gate-defined live/pending | Live                           |
| Production build        | If build-sensitive | Risk-based          | Usually required         | Required          | Required          | Required                  | Provider package if applicable |
| Restart/recovery        | If affected        | Risk-based          | Project-defined          | Affected seams    | Gate-defined      | Required                  | Live service                   |
| Security/privacy        | If affected        | Required            | Required                 | Required          | Required          | Required                  | Required                       |
| Cleanup/canonical proof | Required           | Required            | Required                 | Required          | Required          | Required                  | Required                       |
| Rollback                | If risky           | Risk-based          | Required plan            | Required          | Required          | Required proof            | Provider cleanup/reversal      |
| Release decision        | No                 | No                  | Phase status             | No                | Merge decision    | Yes                       | Provider status                |

## C.1 Evidence reuse rules

- Evidence may be reused only when source, policy, suite/contract versions, environment class, fixture/baseline, dependencies, and gate freshness remain valid.
- A broader gate may consume valid lower-gate evidence but still adds its mandatory nodes and decision rules.
- External live evidence never derives from simulation; local simulation may remain a parallel evidence family.
- Cleanup and canonical-state proof are required for every mutating or resource-owning gate, even when test cases passed.

## APPENDIX D

# Appendix D. Flake, Quarantine, Debt, and Exception Registers

---

## D.1 Flake register template

```
flakeId:
suiteId / caseId / contractId:
firstObserved / lastObserved:
source / policy / environment:
signature:
reproduction attempts:
classification and owner:
risk and release effect:
maximum retries:
remediation plan:
expiry:
closure evidence:
```

## D.2 Quarantine register template

```
quarantineId:
exact scope:
reason and evidence:
risk:
owner / approver:
start / expiry:
release behavior:
compensating controls:
exit criteria:
status:
```

## D.3 Validation-debt template

```
debtId:
missing evidence/capability:
affected contracts/behavior:
risk:
cause:
locally attainable work completed:
compensating evidence:
owner / escalation:
deadline / required future gate:
release effect:
closure receipt:
```

## D.4 Exception register template

```
exceptionId:  
policy clauses:  
requestor / owner / approver:  
reason:  
source/branch/gate/environment scope:  
risk and residual uncertainty:  
compensating controls:  
start / expiry / revocation triggers:  
allowed release status and restrictions:  
closure evidence:
```

## D.5 Register reconciliation

Policy validation compares dates, owners, references, release effects, and closure receipts. Overdue or invalid entries fail closed. Registers are empty by default and visible in project status and release receipts.

## APPENDIX E

# Appendix E. Machine-Readable Policy Schemas

The following examples are conceptual. The repository implementation must publish formal JSON Schemas and tests while preserving the fields and invariants governed by Parts I-III.

## E.1 Suite definition example

```
{
  "schemaVersion": 1,
  "suiteId": "wayfarer.passport.api.authorization",
  "version": 3,
  "tier": 3,
  "owner": "project-wayfarer",
  "contracts": ["passport.owner-projection", "auth.foreign-account-denial"],
  "affectedPaths": ["src/wayfarer/**", "src/app/api/passport/**"],
  "resources": ["sqlite-clone", "node-service"],
  "parallelSafety": "PARALLEL_SAFE_WITH_ISOLATED_DATABASE",
  "risk": "HIGH",
  "releaseGates": ["WAYFARER_PHASE", "MAINLINE", "RELEASE"]
}
```

## E.2 Gate definition example

```
{
  "gateId": "MAINLINE",
  "version": 2,
  "mandatoryContracts": ["repository.policy-valid", "migration.order", "privacy.staged-diff"],
  "mandatorySuiteSelectors": ["impact", "cross-project", "mainline-smoke"],
  "requiredEvidence": ["source", "plan", "results", "cleanup", "remote-parity"],
  "externalPolicy": "record-only-unless-affected",
  "decisionAuthority": "mainline-integrator"
}
```

## E.3 Validation-debt example

```
{
  "debtId": "SL-DEBT-0012",
  "status": "BLOCKED_EXTERNAL",
  "contractId": "private-content.kms-live",
  "owner": "project-sealed-hold",
  "risk": "HIGH",
  "requiredGate": "EXTERNAL_KMS",
  "deadline": "2026-09-30",
  "releaseEffect": "RELEASE_GO_WITH_EXTERNAL_PENDING restricted to non-production KMS"
}
```

## E.4 Schema invariants

- Stable IDs and versions, deterministic canonical JSON, no unresolved references.
- Unknown fields fail closed outside registered extensions.
- Dates use UTC ISO 8601 and expiration is machine-evaluable.
- Paths are repository-relative patterns or registered resource identities.
- Secrets and private content are forbidden.
- Policy digest is computed over canonical files and accepted ADR references.

## APPENDIX F

# Appendix F. Pull Request and Integration Templates

## F.1 Pull request verification section

Behavior changed:  
 Contracts affected:  
 Risk class:  
 Tests added/updated:  
 Negative cases:  
 Sounding Line plan ID:  
 Focused result:  
 Integration/compatibility result:  
 Omitted nodes and reasons:  
 Debt/quarantine/exception IDs:  
 Cleanup receipt:  
 Reviewer authorities:

## F.2 Integration manifest

Candidate branches and commits:  
 Current origin/main:  
 Merge base and migration order:  
 Shared contracts/files:  
 Conflict resolutions:  
 Evidence reused and invalidated:  
 Convergence plan and run IDs:  
 Root failures and repairs:  
 Post-merge proof:  
 Canonical data/resource checks:  
 Final main commit and remote parity:

## F.3 Release candidate checklist

Release artifact/source digest:  
 Policy/runtime/gate versions:  
 Mandatory evidence complete:  
 External gates:  
 Debt/quarantine/exceptions:  
 Security/privacy review:  
 Migration and rollback:  
 Build/browser/accessibility/restart:  
 Cleanup/canonical-state proof:  
 Release decision authority:  
 Final status and restrictions:

### Template rule

Templates make required truth easy to report. They do not permit empty fields, placeholders, or “N/A” without a governed reason.



## APPENDIX G

# Appendix G. Release Decision and Completion Wording Catalog

This catalog governs standard language used in project and release communication. Project-specific prose may be richer, but the evidence status must remain unambiguous.

| Situation                                        | Approved wording                                               | Prohibited implication                      |
|--------------------------------------------------|----------------------------------------------------------------|---------------------------------------------|
| Code exists, limited tests                       | IMPLEMENTED - required validation remains.                     | "Complete."                                 |
| Affected suites pass                             | FOCUSED_VALIDATED for <scope> at <source>.                     | "Release-ready."                            |
| Cross-project seams pass                         | INTEGRATION_VALIDATED for <contracts>.                         | "Merged/mainline" unless true.              |
| Browsers/devices pass                            | COMPATIBILITY_VALIDATED for <matrix>.                          | "All platforms" beyond matrix.              |
| Full release gate passes                         | RELEASE_VALIDATED / RELEASE_GO for <artifact>.                 | Using branch evidence for another artifact. |
| Live provider unavailable after local completion | EXTERNAL_VALIDATION_PENDING / BLOCKED_EXTERNAL for <provider>. | "Live-validated" from mock.                 |
| Required evidence missing                        | RELEASE_INCOMPLETE / NOT_VALIDATED.                            | "Probably safe."                            |
| Independent failure remains                      | RELEASE_NO_GO / FAILED.                                        | Calling descendants failed when blocked.    |
| Evidence mismatch/tamper/cleanup uncertainty     | EVIDENCE_INVALID.                                              | Reusing results anyway.                     |
| Branch complete but not merged                   | on branch ; merged into main: NO.                              | "Users have it."                            |

## G.1 Completion headline example

```
PROJECT <NAME> PHASE <N>
STATUS: INTEGRATION_VALIDATED ON FEATURE BRANCH
MERGED TO MAIN: NO
RELEASE_VALIDATED: NO
EXTERNAL_VALIDATION: <status>
```

Implementation and affected integration evidence are complete for the exact branch source. Mainline merge and release gates remain separate authorized actions.

## APPENDIX H

# Appendix H. Incident Classification and Recovery Playbooks

---

## H.1 Worktree overwrite or destructive cleanup

1. Stop all task admission and cleanup.
2. Preserve commands, runtime roots, manifests, process/resource identity, source status.
3. Identify exact overwritten/deleted paths and whether Git/backup/object copies exist.
4. Do not run broad restore; create forensic copy and obtain repository owner authority.
5. Recover to a new controlled location, verify hashes/history, rerun policy and required gates.
6. Record incident and add path/marker/fault-injection regression.

## H.2 Canonical database mutation

1. Freeze writers and record database path/engine/schema/checksum/row deltas.
2. Determine task/run/account and exact mutations.
3. Backup current state before repair.
4. Use governed targeted rollback/restore or accepted migration repair.
5. Verify canonical state, affected identities, invitations, progress, audit, and secrets.
6. Invalidate evidence produced after contamination and rerun on isolated database.

## H.3 Secret/private data exposure

1. Stop replication and restrict artifact access.
2. Revoke/rotate credentials, sessions, signed URLs, keys as applicable.
3. Classify repository/history/evidence/provider exposure.
4. Remove from active surfaces through authorized redaction/history process.
5. Notify privacy/security owners and affected project authority.
6. Create synthetic regression and scanner/policy improvements.

## H.4 Incorrect release declaration

1. Revoke or amend public/internal release status immediately.
2. Preserve decision receipt and identify missing/invalid evidence.
3. Block further deployment/rollout and assess shipped artifact scope.
4. Run required fresh gate or rollback deployment.
5. Record governance defect and authority failure.
6. Update templates/validation to prevent recurrence.

## H.5 Orphaned runtime/resource collision

1. Inspect durable run state and ownership markers.
2. Do not kill by process name or reclaim solely by PID.
3. Contain ports/databases/storage/provider namespaces with proven identity.
4. Resume or cancel through Part II controller; quarantine ambiguity.
5. Verify cleanup and unrelated processes/resources.
6. Record orphan recovery metrics and adapter correction.

## APPENDIX I

# Appendix I. Performance Budget and Slow-Suite Review Worksheets

## I.1 Budget review worksheet

| Measure         | Current | Target              | Delta | Owner action |
|-----------------|---------|---------------------|-------|--------------|
| Plan generation |         | <10s                |       |              |
| Queue           |         | profile-specific    |       |              |
| Provisioning    |         | baseline            |       |              |
| Setup           |         | suite-specific      |       |              |
| Execution       |         | suite/gate budget   |       |              |
| Teardown        |         | adapter budget      |       |              |
| Critical path   |         | gate target         |       |              |
| Total wall      |         | gate target         |       |              |
| Retry rate      |         | near 0 for critical |       |              |
| Cleanup failure |         | 0 release-blocking  |       |              |

## I.2 Slow-suite analysis prompts

- Which contracts require this tier and environment?
- What portion is queue, provisioning, setup, execution, teardown?
- Can immutable setup be reused safely?
- Can cases be split without order dependence?
- Can shards run with isolated resources?
- Is a cold-start assertion mixed into a warm journey?
- Are waits event-driven?
- Are fixtures larger than the protected boundary needs?
- Does production code have an actual performance defect?
- What accepted evidence proves the optimization preserved coverage?

## I.3 Optimization acceptance

Before duration: median / p95 / critical path  
 After duration: median / p95 / critical path  
 Source and policy identity:  
 Coverage/case/contract comparison:  
 Failure and retry comparison:  
 Isolation and cleanup comparison:  
 Hardware/capacity profile:  
 Accepted savings:  
 Rollback trigger:

## APPENDIX J

# Appendix J. Roles, Authorities, and Separation of Duties

Small teams may assign several roles to one person, but the authority exercised must remain explicit. Automated agents may perform work and produce evidence; they do not silently acquire approval authority.

| Role                             | Primary authority                                                        | Separation requirements                                                       |
|----------------------------------|--------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| Program governance owner         | Accept/amend Parts I-III, final program closure.                         | Material weakening requires independent review and amendment.                 |
| Product/source owner             | Define product behavior and accept semantic changes.                     | Cannot override verification meaning or release evidence.                     |
| Contract owner                   | Define contract version/success/failure and coverage.                    | Must not approve hidden weakening of consumers.                               |
| Test owner                       | Maintain suite, fixtures, metadata, assertions.                          | Critical tests reviewed independently where practical.                        |
| Infrastructure owner             | Runtime, resources, isolation, cleanup, workers.                         | Cannot redefine product success.                                              |
| Gate owner/integrator            | Evaluate gate completeness and convergence.                              | Cannot convert missing/failed evidence into pass.                             |
| Release manager/authorized owner | Issue release decision and restrictions.                                 | Must rely on valid gate receipt; cannot self-grant exception casually.        |
| Security/privacy owner           | Review sensitive fixtures, incidents, redaction, external provider risk. | Required for critical exposure/exception.                                     |
| Codex agent                      | Implement, test, document, commit/push when authorized, report status.   | No self-approved exception/quarantine/release; no destructive/shared actions. |
| CI/runtime worker                | Execute sealed plan nodes and upload evidence.                           | No policy mutation or decision beyond assigned node result.                   |

## APPENDIX K

# Appendix K. Policy Audit and Compliance Scenarios

Part III acceptance includes policy scenarios that deliberately challenge enforcement. Each scenario must produce the governed classification and leave canonical state unchanged.

| Scenario                                                         | Expected result                                                 |
|------------------------------------------------------------------|-----------------------------------------------------------------|
| Behavioral PR has source changes but no affected contracts/tests | Policy validator or review blocks closure.                      |
| Codex adds test.only to make suite pass                          | Policy/static check fails; no completion.                       |
| Timeout doubled without diagnosis                                | Review/policy blocks; evidence does not promote.                |
| One setup node fails before 120 browser cases                    | 1 root failure; descendants cascade-blocked; counts reconcile.  |
| Quarantine entry expires during release plan                     | Gate fails closed or becomes incomplete.                        |
| Debt closed by editing status without receipt                    | Reconciliation reopens/invalidates closure.                     |
| Mock KMS result labeled live-validated                           | Policy violation; provider status corrected; release affected.  |
| Main advances after branch gate                                  | Evidence invalidation analysis and new plan required.           |
| Unowned fixed port declared parallel safe                        | Policy/resource certification fails.                            |
| Release report says complete but branch unmerged                 | Status validator flags contradiction.                           |
| Private invitation token appears in trace                        | Security incident; evidence restricted/invalidated.             |
| Codex launches duplicate durable release run                     | Duplicate-run prevention or policy violation.                   |
| Exception expires after conditional release                      | Release restriction escalates/review required.                  |
| Legacy command and Sounding Line disagree during dual-run        | Cutover blocked; classify selection/result/evidence difference. |
| Cleanup passes cases but leaves MySQL schema                     | Run not complete; cleanup failure and quarantine/recovery.      |

## K.1 Acceptance requirement

Scenarios run under automated tests or controlled drills appropriate to their risk. Results are tied to policy/runtime versions and preserved in the Part III validation record.

## APPENDIX L

# Appendix L. Four-Phase Governance Implementation Map

Part III enforcement develops across all four Sounding Line phases while remaining subordinate to Part I meaning and Part II runtime mechanics.

| Program phase                | Part III governance scope                                                                                                                                           | Exit evidence                                                                             |
|------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| Phase 1 - Take the Soundings | Policy hierarchy, developer/Codex obligations, change-to-test floors, suite/contract/owner/status files, plan-only reviews.                                         | Behavior changes include impact/test statements; policy files validate; unknowns visible. |
| Phase 2 - Open the Channels  | Task-owned runtime requirements, isolation/cleanup policy, parallel certification review, compatibility wrappers.                                                   | Migrated suites use safe resources; no global-lock assumption in new work.                |
| Phase 3 - Read the Current   | Impact-plan enforcement, stale/regression/negative/assertion policies, root/cascade reporting, evidence reuse, slow/flake governance, completion report validation. | Local/Codex/CI affected decisions explain and reconcile; test weakening detected.         |
| Phase 4 - Prove the Passage  | Mainline/release gates, branch protection, distributed trust, release statuses, incidents/exceptions/audits, adoption closure, legacy authority retirement.         | Authoritative reproducible release decisions and final program completion receipt.        |

## L.1 Phase sequencing rules

- Do not enforce impact-selected omission before inventory and policy references are credible.
- Do not mandate broad parallelism before isolation and cleanup are certified.
- Do not retire legacy release authority before dual-run comparison and rollback are proven.
- Introduce completion wording and same-task test obligations early because they do not depend on full runtime cutover.
- Retain conservative all-relevant and emergency serial modes through closure observation.

APPENDIX M

# Appendix M. Part III Normative Completion Checklist

- ☐ Policy hierarchy and conflict resolution are implemented and documented.
- ☐ Developer obligations and same-task source/test enforcement operate.
- ☐ Codex prompt and completion contracts are validated on real major tasks.
- ☐ Change-to-test matrix and negative/regression/stale/assertion policies are machine-checkable where applicable.
- ☐ Release gates are versioned and reference valid suites/contracts/evidence.
- ☐ Mainline and release decision wording cannot exceed evidence.
- ☐ Flake, quarantine, debt, exception, and incident registers validate and reconcile.
- ☐ Evidence retention, privacy, and test-data governance are enforced.
- ☐ Performance budgets and slow-suite reviews are measured and owned.
- ☐ Machine-readable policy files and validator pass negative and compatibility scenarios.
- ☐ Repository status/Ledgerlight/Feature Catalog integration reflects verification truth.
- ☐ Incident and recovery drills prove stop-work, containment, revocation, recovery, fresh proof, and resumption.
- ☐ Adoption stages cover every active suite and legacy authority is retired or wrapped.
- ☐ Training exists for developer, Codex, integrator, release manager, and maintainer.
- ☐ Final program acceptance criteria and completion receipt are satisfied.

## M.1 Companion-document non-duplication

| Part I remains authority for                                                                             | Part II remains authority for                                                                                         | Part III remains authority for                                                                                                                          |
|----------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| Vocabulary, tiers, ownership, contracts, impact meaning, risk, statuses, failures, evidence, confidence. | Graph, scheduler, broker, leases, resources, runtimes, databases, browsers, workers, cleanup, recovery, distribution. | Developer/Codex obligations, change-to-test rules, gates, branch/release policy, instability/debt/exceptions, performance, incidents, rollout, closure. |

**Final review statement**

Part III is accepted only when implementation teams and automated agents can change, test, integrate, and release the repository without inventing obligations, weakening tests, hiding gaps, or using completion language that claims more than the Part I evidence executed by Part II actually proves.

PART III

# R. References and Final Governing Rule

Normative sources include the accepted Project Sounding Line three-document planning record, Part I Software Verification Architecture, Part II Execution Infrastructure and Parallel Runtime, the initial Sounding Line baseline, accepted cross-project governing and completion records, and the current repository implementation at execution time. Historical SHAs and reports document architecture and failure patterns; they never order restoration to the past.

## Primary project references

- Project Sounding Line - Part I: Software Verification Architecture, Version 1.0, Document ID CS-SL-P1-001, July 28, 2026.
- Project Sounding Line - Part II: Execution Infrastructure and Parallel Runtime, Version 1.0, Document ID CS-SL-P2-001, July 28, 2026.
- Project Sounding Line three-document architecture and detailed Part II/Part III planning record, July 28, 2026.
- Project Sounding Line initial governing baseline: The Software Verification and Parallel Test Infrastructure System, Version 1.0.
- Accepted One Voyage, Wayfarer, Sealed Hold, Harborlight, Lanternwake, Landfall, Drydock, Watchglass, True North, Ledgerlight, Feature Catalog, Breakwater, and desktop Companion records.
- Current Kgray44/treasurehuntSoT repository, policy files, test harness, validation scripts, CI, and fetched origin/main at implementation time.

## Standards and technical references

- JSON Schema and canonical JSON practices for policy, plans, registers, receipts, and status files.
- OWASP guidance for secure development, CI/CD, secrets, authorization, logging, supply chain, test data, and incident response.
- WCAG 2.2 and browser accessibility testing practices.
- Git and repository governance practices for branches, worktrees, protected mainline, signed/reproducible release evidence, and auditability.
- Playwright, Vitest, Node.js, Prisma, SQLite, MySQL, browser, Windows, Linux, container, and provider documentation as adopted by accepted implementation records.

## Companion-document boundary

| Document                                                              | Authority                                                                                                                                                                           |
|-----------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Part I - Software Verification Architecture                           | Vocabulary, ownership, contracts, tiers, metadata, impact, risk, status, failure, evidence, and release confidence.                                                                 |
| Part II - Execution Infrastructure and Parallel Runtime               | Graph mechanics, scheduler, broker, resources, baselines, databases, browsers, durable workers, cleanup, recovery, local/CI/distributed execution.                                  |
| Part III - Repository Policy, Codex Governance, and Release Assurance | Human/Codex obligations, change-to-test rules, branch/mainline/release gates, instability/debt/exceptions, retention, performance, incidents, rollout, audit, enforcement, closure. |

### Final governing rule

Part III enforces the proof defined by Part I through the runtime governed by Part II. No developer, Codex task, CI job, integrator, exception, or release message may silently weaken that proof, bypass resource and cleanup ownership, hide missing evidence, or claim a stronger status than the complete reproducible record supports.